

Week1 DSA & OOP

Updated 2026-03-03 14:26 GMT+8
Compiled by Hongfei Yan (2025 spring)
<https://github.com/GMyhf/2026spring-cs201/>

Logs:

- 熟练掌握 Linux 系统、Shell 命令、OJ 测试数据处理、Markdown 编写及 GitHub 使用，是必备的基础技能。
- 2026spring 数算每日选作, https://github.com/GMyhf/2026spring-cs201/blob/main/DSA_problem_list_at_2026spring.md
- LeetCode 热题 100, <https://leetcode.cn/studyplan/top-100-liked/>
包括：哈希，双指针，滑动窗口，矩阵，链表，二叉树，图论，回溯，二分查找，栈，堆，贪心算法，动态规划，技巧
- 小北智学/AI答疑 <https://zx.pku.edu.cn>
- 联合开发团队：北京大学计算机学院闫宏飞老师团队与计算中心共同开发
- 正式上线时间：2025年6月
- 服务对象：面向全校师生开放使用

数据结构与算法B

计算机学院

闫宏飞

合作者：罗熙佑、彭亦男、王嘉林

AI

LLM

本科

算法

春季

秋季

2025-06-13 11:05

440 人

课程微信群：



群聊: 2026spring-cs201: DS
Algo 🌲 算-15 班



该二维码 7 天内 (3月10日前) 有效, 重新进入将更新

引言 数据结构与算法

<https://www.geeksforgeeks.org/learn-data-structures-and-algorithms-dsa-tutorial/?ref=outind>

What is DSA?

数据结构与算法 (Data Structures and Algorithms, 简称 DSA) 是两个独立但又紧密关联的计算机科学领域的结合。它被广泛认为是每一位计算机专业学生必须掌握的核心技能之一。

在实践中我们常常发现, 对数据结构与算法有深入理解的程序员, 通常具备更强的编程能力和问题解决能力。

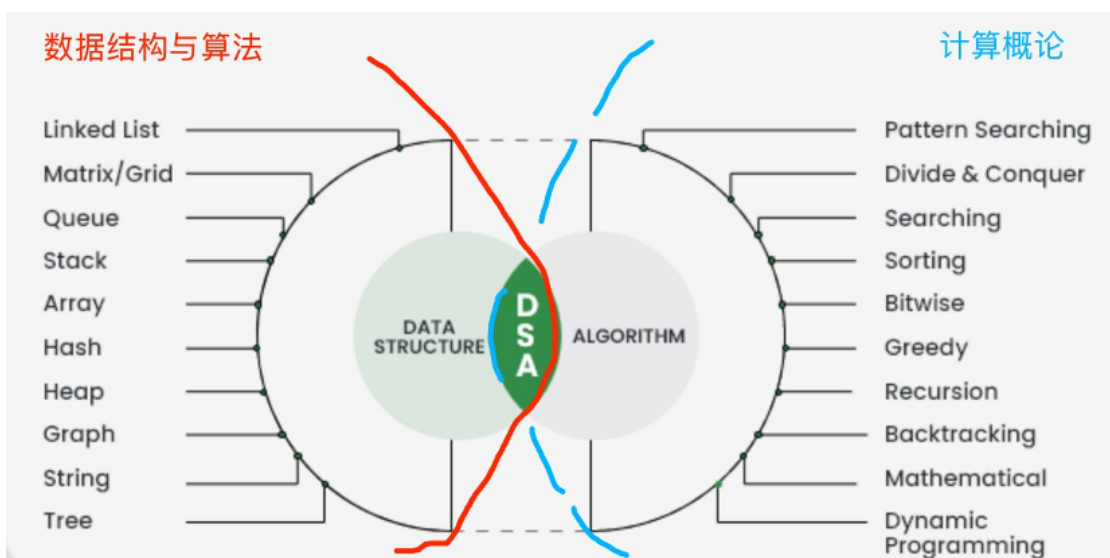
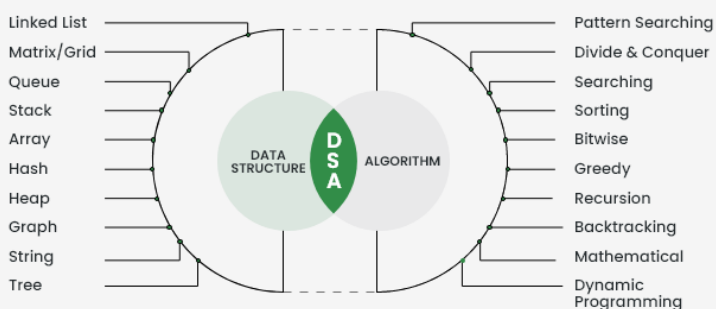
掌握 DSA, 不仅是提升编程能力的关键, 更是通往优秀软件工程师之路的基石。正因如此, 他们往往能在激烈的求职竞争中脱颖而出, 成功通过各大科技公司的技术面试。

DSA is defined as a combination of two separate yet interrelated topics – Data Structure and Algorithms. DSA is one of the most important skills that every computer science student must have. It is often seen that people with good knowledge of these technologies are better programmers than others and thus, crack the interviews of almost every tech giant.



Data Structures & Algorithm

DSA



⚠ 重要提示：算法是前序《计算概论》（Algorithms and Data Structures, 简称 ADS）课程的核心内容

ADS课程涵盖以下四类关键算法：

- 贪心算法（Greedy）
- 递归与回溯（Recursion / Backtracking）
- 动态规划（Dynamic Programming, DP）
- 搜索算法（Searching）

如果你在递归类题目上练习不足，建议针对性地多做相关题目以巩固基础。

遇到不会的题目怎么办？欢迎随时在课程群内提问！

学习建议：

1. 递归（Recursion）

是算法设计中的核心基础技能，建议优先掌握。

参考资料：递归、回溯、并查集, https://github.com/GMyhf/2025fall-cs101/blob/main/20251028_ADS_week08-09_recursion_backtracking.md

2. 队列与广度优先搜索（BFS）

队列在广度优先搜索中有广泛应用。

参考资料：搜索专题, https://github.com/GMyhf/2025fall-cs101/blob/main/20251125_ADS_week12_searching.md

3. 其他常用算法技巧

即使尚未系统学习，也无需担心。遇到相关问题时逐步掌握即可。常见技巧包括：

- 双指针（链表中的“快慢指针”需重点掌握）
- 单调栈
- 二分查找
- 并查集
- 滑动窗口
- 懒删除（Lazy Deletion）

多数技巧通过 1~2 道典型题目即可理解其基本原理，但要熟练运用仍需持续练习。

4. 面向对象编程（OOP）

属于语言语法层面的内容，建议参考：

菜鸟教程：Python3 面向对象, <https://www.runoob.com/python3/python3-class.html>

Python3 面向对象

Python3 命名空间/作用域

Python3 标准库概览

Python3 实例

Python 测验

Python3 高级教程

Python3 正则表达式

Python3 CGI编程

Python3 MySQL(mysql-connector)

Python3 MySQL(PyMySQL)

Python3 网络编程

Python3 SMTP发送邮件

Python3 多线程

Python3 XML 解析

Python3 JSON

Python3 日期和时间

Python3 内置函数

Python3 MongoDB

Python3 urllib

Python uWSGI 安装配置

Python3 pip

Python3 operator

Python math

Python3 面向对象

Python从设计之初就已经是一门面向对象的语言，正因为如此，在Python中创建一个类和对象是很容易的。本章节我们将详细介绍Python的面向对象编程。

如果你以前没有接触过面向对象的编程语言，那你可能需要先了解一些面向对象语言的一些基本特征，在头脑里形成一个基本的面向对象的概念，这样有助于你更容易的学习Python的面向对象编程。

接下来我们先来简单的了解下面面向对象的一些基本特征。

面向对象技术简介

- **类(Class):** 用来描述具有相同的属性和方法的对象的集合。它定义了该集合中每个对象所共有的属性和方法。对象是类的实例。
- **方法:** 类中定义的函数。
- **类变量:** 类变量在整个实例化的对象中是公用的。类变量定义在类中且在函数体之外。类变量通常不作为实例变量使用。
- **数据成员:** 类变量或者实例变量用于处理类及其实例对象的相关的数据。
- **方法重写:** 如果从父类继承的方法不能满足子类的需求，可以对其进行改写，这个过程叫方法的覆盖（override），也称为方法的重写。
- **局部变量:** 定义在方法中的变量，只作用于当前实例的类。
- **实例变量:** 在类的声明中，属性是用变量来表示的，这种变量就称为实例变量，实例变量就是一个用 self 修饰的变量。
- **继承:** 即一个派生类（derived class）继承基类（base class）的字段和方法。继承也允许把一个派生类的对象作为一个基类对象对待。例如，有这样一个设计：一个Dog类型的对象派生自Animal类，这是模拟“是一个（is-a）”关系（例图，Dog是一个Animal）。
- **实例化:** 创建一个类的实例，类的具体对象。
- **对象:** 通过类定义的数据结构实例。对象包括两个数据成员（类变量和实例变量）和方法。

和其它编程语言相比，Python 在尽可能不增加新的语法和语义的情况下加入了类机制。

Python中的类提供了面向对象编程的所有基本功能：类的继承机制允许多个基类，派生类可以覆盖基类中的任何方法，方法中可以调用基类中的同名方法。

对象可以包含任意数量和类型的数据。

What is Data Structure?

数据结构是指一种特定的存储和组织数据的方式，旨在我们的设备中高效且有效地管理和利用数据。采用数据结构的核心理念在于最小化时间和空间复杂度，即通过占用尽可能少的内存空间并以最短的时间来执行数据操作，从而实现高效的性能。

A data structure is defined as a particular way of storing and organizing data in our devices to use the data efficiently and effectively. The main idea behind using data structures is to minimize the time and space complexities. An efficient data structure takes minimum memory space and requires minimum time to execute the data.

What is Algorithm?

算法是指为了解决特定类型的问题或执行某种特定计算而设计的一系列明确定义的步骤。用更简单的话来说，算法就是一组按部就班的操作，通过这些操作来完成一项具体的任务。

Algorithm is defined as a process or set of well-defined instructions that are typically used to solve a particular group of problems or perform a specific type of calculation. To explain in simpler terms, it is a set of operations performed in a step-by-step manner to execute a task.

How to start learning DSA?

首先要做的是将整个学习过程分解为一系列需按顺序完成的小任务。从零开始系统地学习数据结构与算法（DSA）的过程可以分为以下四个阶段：

1. **理解时间和空间复杂度**：掌握评估算法效率的关键概念。
2. **学习各数据结构的基础**：熟悉不同数据结构的特点和使用场景。
3. **掌握算法的基础知识**：了解常用算法的工作原理及其应用。
4. **练习DSA相关的题目**：通过实践巩固所学知识，提高解决问题的能力。

The first and foremost thing is dividing the total procedure into little pieces which need to be done sequentially. The complete process to learn DSA from scratch can be broken into 4 parts:

1. Learn about Time and Space complexities
2. Learn the basics of individual Data Structures
3. Learn the basics of Algorithms
4. Practice Problems on DSA

逻辑视角、物理视角和数据结构

计算机科学的核心研究对象是**问题、解决问题的过程，以及通过该过程得到的解决方案**。给定一个问题，计算机科学家的目标是构建一个能够逐步解决该问题的**算法**。算法是一种有限步骤的过程，遵循这一过程即可得到问题的解答。换言之，算法就是问题的解决方案。

从这个角度看，可以认为计算机科学就是研究算法的学科。但需要注意的是，并非所有问题都存在解法。尽管这一话题超出了本书的范围，但对学习计算机科学的人来说，认识到某些问题无解是至关重要的。综合来看，计算机科学可以被更完整地定义为：**研究问题及其解决方案，以及研究那些目前无法解决的问题的学科**。

除了问题与解决方案，计算机科学还强调**抽象**。抽象思维使我们能够分别从逻辑视角和物理视角来理解问题与解决方案。

例如，想象你每天开车去上班或上学。作为司机，你只需会使用方向盘、油门、刹车、档位与点火钥匙，这是从**逻辑视角**（接口）看待汽车。而修车工则必须理解发动机、变速器、温控系统等细节，这是**物理视角**。

同样地，大多数计算机使用者只需通过界面完成写作、收发邮件、浏览网页、听音乐、存储照片和玩游戏等任务，而无需了解其背后的技术细节。但计算机科学家、程序员、系统管理员等则必须理解操作系统、网络协议、脚本编写等底层机制。

抽象的本质在于：用户只需掌握**接口**的用法，而无需了解实现的细节。举例来说，Python 的 `math` 模块提供了一个计算平方根的函数：

```
1 >>> import math
2 >>> math.sqrt(16)
3 4.0
4 >>>
```

这是一个典型的**过程抽象**示例。我们并不需要知道平方根的计算过程，只需了解函数名、输入参数与返回值即可。函数的实现细节被隐藏在“黑盒”中。

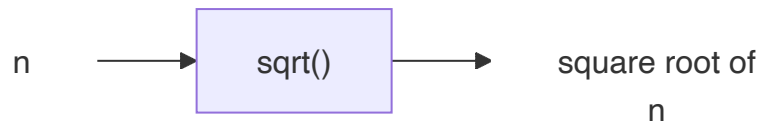


图1-1 过程抽象

1 为何学习数据结构与抽象数据类型

在面对复杂问题时，计算机科学家常借助**抽象**来降低认知负担，专注于问题的本质，而非陷入实现的细枝末节。通过对现实问题进行建模，能够更高效地设计和实现解决方案。其中，数据模型在算法设计中扮演着关键角色，它帮助我们一致、清晰地描述算法所操作的数据。

如前所述，**过程抽象**通过隐藏功能的具体实现，使开发者能够从更高层次理解程序行为。类似地，**数据抽象**的核心思想是将数据的使用与其底层实现分离。**抽象数据类型**（Abstract Data Type，简称 ADT）正是这一思想的体现：它从逻辑上定义了数据的结构及其可执行的操作，而无需暴露具体的实现方式。换言之，我们只需关注“数据代表什么”和“可以对数据做什么”，而无需关心“数据是如何存储或操作的”。

这种抽象本质上是一种**封装**，其核心原则是**信息隐藏**——将实现细节隔离在接口之后，仅向用户提供一组明确的操作接口。这不仅提升了代码的可维护性，也增强了系统的模块化程度。

图1-1 展示了抽象数据类型的基本原理。用户通过 ADT 提供的接口与数据交互，而具体的实现则被封装在内部，对用户透明。ADT 充当了用户与数据之间的“外壳”，屏蔽了复杂性，使用户能够专注于问题求解本身。

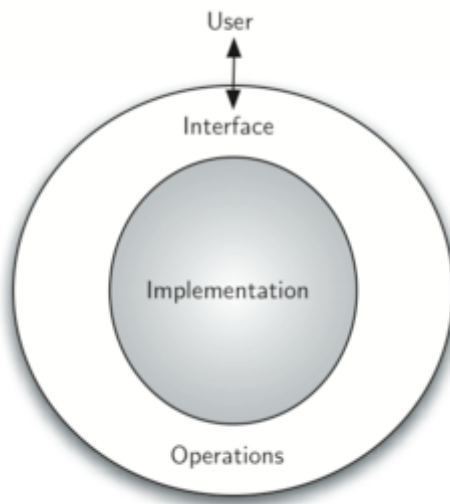


图1-1 抽象数据类型

抽象数据类型的实现通常被称为数据结构。它是连接逻辑视图与物理视图的桥梁。一方面，数据结构需体现 ADT 的逻辑特性（如栈的“后进先出”或队列的“先进先出”）；另一方面，它也决定了这些特性如何在计算机中具体实现（例如使用数组或链表）。通过这种分离，可以在不暴露底层细节的前提下，构建复杂而高效的数据模型。

这种逻辑与物理的分离赋予了数据模型实现无关性（implementation independence）。由于同一个 ADT 往往可以通过多种数据结构实现（如栈可用数组或链表实现），实现的灵活性使得程序员可以在不改变用户接口的前提下，根据性能需求或环境约束切换底层实现。用户则可以始终专注于问题本身的逻辑，而不受实现变更的影响。

The implementation of an abstract data type, often referred to as a **data structure**, will require that we provide a physical view of the data using some collection of programming constructs and primitive data types. As we discussed earlier, the separation of these two perspectives will allow us to define the complex data models for our problems without giving any indication as to the details of how the model will actually be built. This provides an **implementation-independent** view of the data. Since there will usually be many different ways to implement an abstract data type, this implementation independence allows the programmer to switch the details of the implementation without changing the way the user of the data interacts with it. The user can remain focused on the problem-solving process.

术语辨析：物理视图、逻辑视图与数据结构

在计算机科学与软件工程中，“物理视图”、“逻辑视图”（或“数据视图”）和“数据结构”是三个密切相关但含义不同的概念：

物理视图 (Physical View)：

- 指数据在内存或存储设备中的实际组织方式，包括数据的存储布局、访问机制、指针结构等底层细节。例如，链表中节点通过指针链接，数组元素在内存中连续存放，这些都属于物理视图的范畴。

逻辑视图（也称为数据视图, Logical View / Data View）：

- 指从用户或应用程序角度所看到的数据组织形式。它是 ADT 的核心，定义了数据的抽象行为和操作接口，而不涉及实现。例如，栈的逻辑视图只关注 `push` 和 `pop` 操作的行为规范，而不关心其内部是用数组还是链表实现。

数据结构 (Data Structure)：

- 数据结构是用来组织、管理和存储数据的方式，以便能够高效地访问和修改数据。它是物理视图和逻辑视图之间的桥梁，因为数据结构既定义了数据的逻辑特性（如栈或队列的操作），也决定了这些特性的具体实现方式（如使用数组或链表）。其选择直接影响到程序的性能和复杂度。

原文中的 "physical view of the data" 更贴近于“数据的物理视图”，即关注的是数据在底层硬件上的表示和存储方式；而“implementation-independent view of the data” 则更倾向于描述“逻辑视图”或“数据视图”，强调的是与具体实现无关的数据抽象。

数据结构的分类

数据结构通常从两个维度进行分类：**逻辑结构与物理结构**。

```

1  数据结构
2  └─ 逻辑结构
3  │   └─ 集合结构：数据元素间无特定关系
4  │   └─ 线性结构：数据元素呈线性排列
5  │       └─ 线性表（如数组、链表）
6  │       └─ 栈（Stack）
7  │       └─ 队列（Queue）
8  │       └─ 串（String）
9  │   └─ 非线性结构：数据元素间存在多对多或层次关系
10 │       └─ 树形结构（如二叉树、B树、堆）
11 │       └─ 图结构（Graph）
12 └─ 物理结构（存储结构）
13     └─ 顺序存储：数据元素在内存中连续存放（如数组）
14     └─ 链式存储：通过指针链接非连续的存储单元（如链表）
15     └─ 索引存储（Indexing）：建立索引表以加快查找，适用于有序数据和范围查询（如 B树、
      B+树）
16     └─ 散列存储（Hashing）：通过哈希函数直接定位数据，适用于快速键值查找
  
```

通过理解抽象数据类型与数据结构之间的关系，我们能够更系统地设计和选择合适的组织方式，从而在解决实际问题时兼顾**逻辑清晰性**与**实现高效性**。这正是学习数据结构与抽象数据类型的根本意义所在。

编程题目

06640: 倒排索引

data structures, <http://cs101.openjudge.cn/practice/06640/>

04093: 倒排索引查询

data structures, <http://cs101.openjudge.cn/practice/04093/>

在信息检索Information Retrieval领域，搜索引擎的工作原理是抓取网页，每个网页是一个doc，处理成token的序列。为了检索时候快速响应用户输入的查询token，所以要建token -> doc的倒排索引。

document -> token, 是正排。token -> doc_i, doc_j, ... 是倒排。名字就是这样来的。

Q: 二叉树 (Binary Tree) 和 B 树 (B-Tree)

二叉树 (Binary Tree) 和 B 树 (B-Tree) 是两种常见的树形数据结构, 它们在计算机科学中被广泛用于组织和管理数据。虽然都属于“树”结构, 但它们的结构特点、应用场景和性能特征有很大不同。下面是对它们的详细对比:

一、二叉树 (Binary Tree)

定义:

- 每个节点最多有两个子节点: 左子节点和右子节点。
- 子树有左右之分, 次序不能颠倒。

常见类型:

- 普通二叉树: 无特殊限制。
- 满二叉树: 除叶子节点外, 每个节点都有两个子节点。
- 完全二叉树: 除了最后一层, 其他层都被填满, 且最后一层节点靠左对齐。
- 二叉搜索树 (BST): 左子树所有节点值 < 根节点值 < 右子树所有节点值。
- 平衡二叉树 (如 AVL 树、红黑树): 通过旋转等操作保持树的高度平衡, 以保证查找效率。

特点:

- 适合内存中的数据结构。
- 查找、插入、删除平均时间复杂度为 $O(\log n)$ (在平衡情况下)。
- 不适合磁盘存储 (因为每次访问一个节点可能需要一次 I/O 操作)。

应用场景:

- 表达式解析 (表达式树)
- 决策树
- 二叉搜索树用于实现有序映射 (如 C++ 的 `std::map` 使用红黑树)
- 哈夫曼编码

二、B 树 (B-Tree)

定义:

- 是一种多路平衡搜索树, 专为磁盘或其他直接存取辅助存储设备设计。
- 每个节点可以有多个子节点 (通常几十甚至上百个), 具体数量由“阶数” (order) 决定。
- 所有叶子节点位于同一层, 保证平衡。

结构特点 (以 m 阶 B 树为例):

- 每个节点最多有 m 个子节点。

- 每个内部节点（非根非叶）至少有 $\lceil m/2 \rceil$ 个子节点。
- 根节点至少有 2 个子节点（除非是叶子）。
- 节点中关键字数量 = 子节点数 - 1。
- 关键字按升序排列，子树之间的值范围由关键字界定。

优点：

- **减少磁盘 I/O 次数**：因为每个节点可以存储大量关键字，树的高度非常低。
- **自平衡**：插入/删除时通过分裂或合并保持平衡。
- 高效支持范围查询。

应用场景：

- 数据库索引（如 MySQL 的 InnoDB 引擎使用 B+ 树，B 树的变种）
- 文件系统（如 NTFS、ext4）
- 大规模数据存储系统

三、主要区别总结

特性	二叉树（如 BST）	B 树
子节点数量	最多 2 个	多个（由阶数决定）
平衡性	不一定平衡（需 AVL/RB）	自动保持平衡
存储位置	主要用于内存	专为磁盘/外存优化
I/O 效率	低（每层一次 I/O）	高（每节点含多关键字）
树高度	较高 ($O(\log_2 n)$)	很低 ($O(\log_m n)$, m 很大)
典型用途	算法、内存数据结构	数据库、文件系统索引

四、补充：B+ 树（B-Tree 的常见变种）

- 所有数据只存储在**叶子节点**，内部节点仅作索引。
- 叶子节点通过指针连接，便于范围扫描。
- 现代数据库几乎都使用 **B+ 树**而非原始 **B 树**。

如果你有特定的应用场景（比如“为什么数据库不用二叉树而用 B 树？”），或者想了解某一种树的实现细节（如插入/删除过程），欢迎继续提问！

2 为何学习算法

计算机科学的学习依赖于经验：既要观察他人如何解决问题，也要亲自实践。接触多样化的算法与问题解决方法，有助于在新情境下灵活应对。

不同算法在效率和资源消耗上可能有巨大差异。例如，两个平方根算法都能给出正确答案，但一个可能比另一个快十倍。学习算法分析技巧，能够帮助我们比较算法优劣，并在实践中选择最合适的方案。

有些问题在合理的时间内无法被任何算法解决。这就要求计算机科学家能够区分：

- 可解的问题；
- 无解的问题；
- 有解但代价过高的问题。

算法学习的意义不仅在于“找到解”，更在于评估解的优劣。这是一个需要持续练习与经验积累的能力。

1 OOP及Python基础

本节将通过更具体的示例来说明前面介绍的思想。

Python是一门现代化、易于学习的面向对象编程语言。它不仅拥有强大的内建数据类型，还提供了简洁易用的控制语句。作为解释型语言，Python无需编译即可直接运行代码。学习时，只需观察交互式会话的输入与输出即可。解释器会显示提示符 `>>>`，等待输入，并立即执行代码。

例如，以下会话展示了提示符、`print` 函数、运行结果，以及下一个提示符：

```
1 >>> print("Algorithms and Data Structures")
2 >>> Algorithms and Data Structures
3 >>>
```

1.1 数据

Python支持面向对象编程（OOP）。这意味着在Python中，数据被视为问题求解的核心。

在OOP中，类（class）用来描述数据的构成（状态）以及数据可以执行的操作（行为）。通过类实例化得到的对象（object）就是数据的具体表现形式。由于使用者只需关注对象的状态与行为，而无需了解其底层实现，类也可以被看作是一种抽象数据类型（ADT）。

1.1.1 内建原子数据类型

Python提供了多种基础数据类型：

- **整数与浮点数**：由 `int` 和 `float` 类实现，支持常见的数学运算符：
`+`、`-`、`*`、`/`、`**`（幂）、`%`（取模）、`//`（整除）。
注意：两个整数相除时结果为浮点数，而 `//` 会截断小数部分，仅返回整数部分。
- **布尔类型**：由 `bool` 类实现，只有 `True` 和 `False` 两个值。常用逻辑运算符包括：`and`、`or`、`not`。
布尔值通常作为比较运算（如 `==`、`>`）的结果使用。逻辑运算与关系运算结合，可以表达复杂条件。

表1-1展示了常见的关系与逻辑运算符：

逻辑运算符（and, or, not）的主要任务就是用来连接多个关系运算符（比较表达式），从而构建出复杂的判断条件。

Operation Name	Operator	Explanation
less than	<	Less than operator
greater than	>	Greater than operator
less than or equal	<=	Less than or equal to operator
greater than or equal	>=	Greater than or equal to operator
equal	==	Equality operator
not equal	!=	Not equal operator
logical and	<i>and</i>	Both operands True for result to be True
logical or	<i>or</i>	One or the other operand is True for the result to be True
logical not	<i>not</i>	Negates the truth value, False becomes True, True becomes False

标识符在程序中用作名字。

在Python中，标识符必须以字母或下划线开头，可以包含字母、数字或下划线，并且区分大小写。采用有意义的命名是良好的编程习惯，有助于代码可读性。

变量在首次出现在赋值语句左侧时被创建。赋值语句将名字与数据绑定，变量存储的是数据的引用，而非数据本身。这体现了Python的动态特性：同一个变量在不同时间可以引用不同类型的对象。

教学目标：深化对经典数据结构的理解与应用，提升算法设计与分析能力，培养工程化编程思维和代码质量意识。

教学方式：采用“案例教学+实践训练”的教学模式，通过151余道综合性编程题目进行实践训练，强化算法应用能力。

考核重点：算法效率（时间复杂度、空间复杂度），程序设计规范性，问题建模与解决能力。

代码要求：遵循PEP 8代码规范，鼓励采用面向对象编程（OOP）范式，强调代码的可测试性和可扩展性。

Q: Python 操作符优先级，与 C++ 操作符优先级的不同？比如：位运算（bit manipulation）、移位运算，与==关系。

Python 和 C++ 在操作符优先级上有很多相似之处（比如乘除高于加减，算术运算高于移位运算等），但它们在位运算（&, |, ^）、移位运算（<<, >>）与关系运算（==, <）之间的优先级设计上，存在着极其致命的差异。

很多有 C++ 经验的程序员在写 Python 时，或者反过来时，最容易在这些地方踩坑。以下是核心差异和详细对比：

1. 最致命的区别：位运算（&, |, ^）与 ==

这是 C/C++ 历史上最著名的设计缺陷之一（Dennis Ritchie 本人也承认），而 Python 作为后来者修复了这个问题。

- 在 C++ 中：`==` 的优先级 高于 位运算符（`&`, `|`, `^`）。
- 在 Python 中：位运算符（`&`, `|`, `^`）的优先级 高于 `==`。

踩坑示例：检查一个数字的最低位是否为 0。

```
1 // C++ 代码
2 int a = 2;
3 if (a & 1 == 0) {
4     // 这里的逻辑是错误的!
5     // 实际执行顺序: a & (1 == 0) => 2 & 0 => 0 (false)
6 }
7 // C++ 中必须加括号: if ((a & 1) == 0)
```

```
1 # Python 代码
2 a = 2
3 if a & 1 == 0:
4     # 这里的逻辑是完全正确的!
5     # 实际执行顺序: (a & 1) == 0 => 0 == 0 => True
6     pass
```

2. 移位运算 (<<, >>) 与 ==

在这一点上，Python 和 C++ 是一致的。

两者中，移位运算符的优先级都 高于 关系运算符（`==`, `<`, `>` 等），也高于位逻辑运算符（`&`, `|`, `^`）。

示例对比：

```
1 // C++: 1 << 2 == 4 等价于 (1 << 2) == 4 => 4 == 4 => true
2 // C++: 1 << 1 & 2 等价于 (1 << 1) & 2 => 2 & 2 => 2
```

```
1 # Python: 1 << 2 == 4 等价于 (1 << 2) == 4 => True
2 # Python: 1 << 1 & 2 等价于 (1 << 1) & 2 => 2
```

注意：在这两种语言中，加减法（`+`, `-`）的优先级都高于移位运算（`<<`, `>>`）。所以 `1 << 1 + 1` 在两种语言中都会被计算为 `1 << (1 + 1)` 即 `4`。

3. 比较运算符内部的优先级与链式比较 (< 与 ==)

这是另一个巨大的差异点。

- 在 C++ 中：大小比较（`<`, `>`, `<=`, `>=`）的优先级 高于 相等比较（`==`, `!=`）。而且 C++ 不支持数学意义上的链式比较。
- 在 Python 中：所有比较运算符（`<`, `>`, `==`, `!=`, `is`, `in`）的优先级完全相同，并且 Python 支持链式比较。

踩坑示例：`2 < 1 == 0`

```

1 // C++ 代码
2 bool res = 2 < 1 == 0;
3 // 实际执行: (2 < 1) == 0 => false == 0 => 0 == 0 => true

```

```

1 # Python 代码
2 res = 2 < 1 == 0
3 # 实际执行: 2 < 1 and 1 == 0 => False and False => False

```

总结：优先级顺序天梯图对比

为了直观对比，我们把这几个关键操作符从高到低排列：

优先级 (由高到低)	C++ 运算符顺序	Python 运算符顺序	核心差异说明
高	算术加减 (+, -)	算术加减 (+, -)	相同。都高于移位。
↓	移位 (<<, >>)	移位 (<<, >>)	相同。都高于比较。
↓	大小比较 (<, >)	位运算 (&)	⚠ Python 将位运算放在比较运算之上。
↓	相等比较 (==, !=)	位运算 (^)	
↓	位运算 (&)	位运算 (\)	⚠ C++ 将位运算放在了 == 和 < 之下。
↓	位运算 (^)	比较运算 (<, ==, in)	⚠ Python 中所有比较运算同级，且可链式展开。
↓	位运算 (\)	逻辑非 (not)	
↓	逻辑与 (&&)	逻辑与 (and)	相同。逻辑运算都在位运算和比较运算之下。
低	逻辑或 (\ \)	逻辑或 (or)	

给程序员的建议：

1. 写 C++ 时：只要位运算符 (&, \|, ^) 碰到了 == 或 !=，必须无脑加括号（例如 `if ((x & mask) == 0)`）。
2. 写 Python 时：虽然你可以省去括号 `if x & mask == 0`，但为了可读性以及不让阅读代码的 C++ 程序员抓狂，仍然建议显式加上括号。

1.1.2 内建集合数据类型

除数值与布尔类型外，Python还内置了多种集合类。它们可以分为两大类：

- **有序集合**：列表（list）、字符串（string）、元组（tuple）
- **无序集合**：集合（set）、字典（dict）

下面逐一介绍。

(1) 列表（list）

列表是零个或多个数据对象引用的**有序集合**，通过在方括号 `[]` 中用逗号分隔一系列值来表示，例如：

```
1 >>> myList = [1, "hello", 3.14]
2 >>> emptyList = []
```

列表是**异构**的，也就是说，元素可以属于不同的数据类型。由于列表有序，它支持一系列操作（表1-2）。

表1-2 Python序列支持的基本操作

Operation Name	Operator	Explanation
indexing	<code>[]</code>	访问指定位置的元素（下标从0开始）
concatenation	<code>+</code>	拼接两个序列
repetition	<code>*</code>	重复序列若干次
membership	<code>in</code>	判断元素是否在序列中
length	<code>len</code>	返回序列长度
slicing	<code>[:]</code>	截取子序列

例如：

```
1 >>> myList = [0] * 6
2 >>> myList
3 [0, 0, 0, 0, 0, 0]
```

需要⚠️：**重复运算符复制的是引用，而不是数据本身**：

```
1 >>> myList = [1,2,3,4]
2 >>> A = [myList]*3
3 >>> A
4 [[1, 2, 3, 4], [1, 2, 3, 4], [1, 2, 3, 4]]
5 >>> myList[2] = 45
6 >>> A
7 [[1, 2, 45, 4], [1, 2, 45, 4], [1, 2, 45, 4]]
```

此时，`A` 中三个子列表实际上都指向同一个 `myList`。

Python还为列表提供了丰富的方法（表1-3）：

表1-3 Python列表常用方法

Method Name	Use	Explanation
<code>append</code>	<code>alist.append(item)</code>	在末尾添加新元素
<code>insert</code>	<code>alist.insert(i,item)</code>	在指定位置插入元素
<code>pop</code>	<code>alist.pop()</code>	移除并返回最后一个元素
<code>pop</code>	<code>alist.pop(i)</code>	移除并返回第 <code>i</code> 个元素
<code>sort</code>	<code>alist.sort()</code>	就地排序
<code>reverse</code>	<code>alist.reverse()</code>	就地反转顺序
<code>del</code>	<code>del alist[i]</code>	删除第 <code>i</code> 个元素
<code>index</code>	<code>alist.index(item)</code>	返回首次出现 <code>item</code> 的下标
<code>count</code>	<code>alist.count(item)</code>	返回 <code>item</code> 出现次数
<code>remove</code>	<code>alist.remove(item)</code>	移除首次出现的 <code>item</code>

像pop这样的方法在返回值的同时也会修改列表的内容，`reverse`等方法则仅修改列表而不返回任何值。pop默认返回并删除列表的最后一个元素，但是也可以用来返回并删除特定的元素。这些方法默认下标从0开始。你也会注意到那个熟悉的句点符号，它被用来调用某个对象的方法。

range是一个常见的Python函数，我们常把它与列表放在一起讨论。range会生成一个代表值序列的范围对象。使用list函数，能够以列表形式看到范围对象的值。

范围对象表示整数序列。默认情况下，它从0开始。如果提供更多的参数，它可以在特定的点开始和结束，并且跳过中间的值。在第一个例子中，range(10)从0开始并且一直到9为止（不包含10）；在第二个例子中，range(5,10)从5开始并且到9为止（不包含10）；range(5,10,2)的结果类似，但是元素的间隔变成了2（10还是没有包含在其中）。

Matrices 矩阵

在学习编程的过程中，经常遇到输入的数据是矩阵的形式，所以我们首先来明确矩阵的概念。

知识点：矩阵

这段矩阵知识点的讲解，借鉴自《数学要素》的1.4和1.5节，作者：姜伟生，2023-06-01出版。

万物皆数。
All is Number.

——毕达哥拉斯(Pythagoras) | 古希腊哲学家、数学家 | 570 B.C.—
495 B.C.

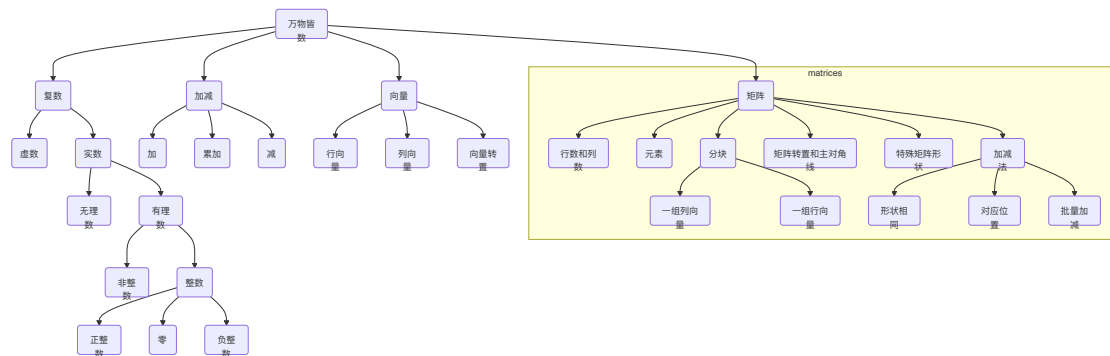


图1 数的结构

向量：数字排成行、列

向量、矩阵等线性代数概念对于数据科学和机器学习至关重要。在机器学习中，数据几乎都以矩阵形式存储、运算。毫不夸张地说，没有线性代数就没有现代计算机运算。逐渐地，大家会发现算数、代数、解析几何、微积分、概率统计、优化方法并不是一个个孤岛，而线性代数正是连接它们的重要桥梁之一。

行向量、列向量

若干数字排成一行或一列，并且用中括号括起来，得到的数组叫作向量(vector)。

排成一行的叫作行向量(row vector)，排成一列的叫作列向量(column vector)。

通俗地讲，行向量就是表格的一行数字，列向量就是表格的一列数字。以下两例分别展示了行向量和列向量，即

$$[1 \quad 2 \quad 3]_{1 \times 3}, \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}_{3 \times 1} \quad (1)$$

式(1)中，下角标“1×3”代表“1行、3列”，“3×1”代表“3行、1列”。

转置

转置符号为上标“T”。行向量转置(transpose)可得到列向量；同理，列向量转置可得到行向量。举例如下，有

$$[1 \quad 2 \quad 3]^T = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}, \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}^T = [1 \quad 2 \quad 3] \quad (2)$$

矩阵：数字排列成长方形

矩阵(matrix)将一系列数字以长方形方式排列，如

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}_{2 \times 3}, \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix}_{3 \times 2}, \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}_{2 \times 2} \quad (3)$$

通俗地讲，矩阵将数字排列成表格，有行、有列。式(3)给出了三个矩阵，形状分别是2行3列、3行2列和2行2列。

通常用大写字母代表矩阵，比如矩阵A和矩阵B。

图2所示为一个n×D矩阵X。n是矩阵的行数(number of rows in the matrix)，D是矩阵的列数

(number of columns in the matrix)。X可以展开写成表格形式，即

$$X_{n \times D} = \begin{bmatrix} x_{1,1} & x_{1,2} & \dots & x_{1,D} \\ x_{2,1} & x_{2,2} & \dots & x_{2,D} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n,1} & x_{n,2} & \dots & x_{n,D} \end{bmatrix}$$

(4)

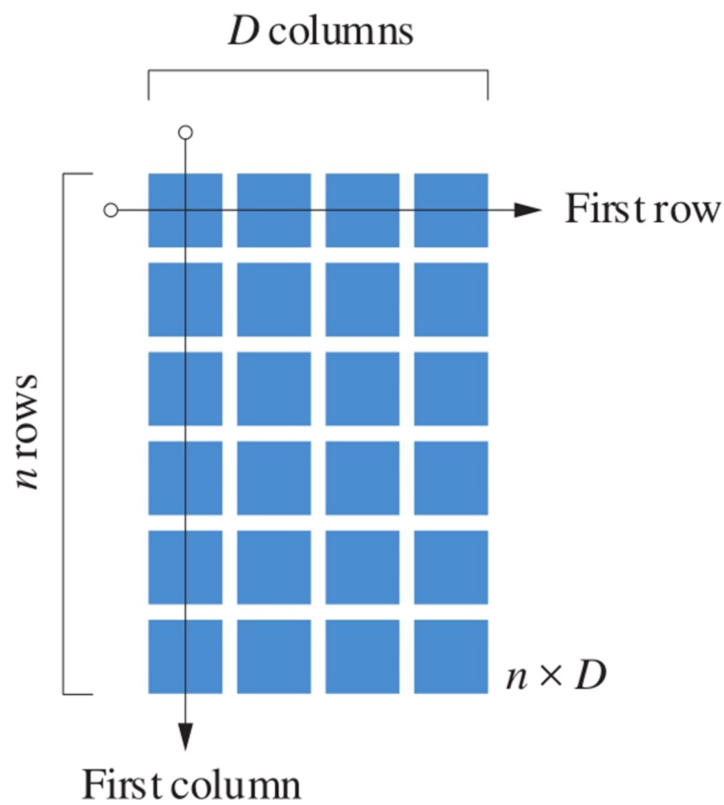


图2 $n \times D$ 矩阵X

再次强调：先说行号，再说列号。数据矩阵一般采用大写X表达。

矩阵X中，元素(element) $x_{i,j}$ 被称作*i,j*元素 (*i j entry*或*i j element*)，也可以说 $x_{i,j}$ 出现在*i*行*j*列 (appears in row *i* and column *j*)。比如， $x_{n,1}$ 是矩阵X的第*n*行、第1列元素。

表1 总结了如何用英文读矩阵和矩阵元素。

表1 矩阵有关英文表达

数学表达	英文表达
$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$	Two by two matrix, first row one two, second row three four
$\begin{bmatrix} a_{1,1} & a_{1,2} & \cdots & a_{1,n} \\ a_{2,1} & a_{2,2} & \cdots & a_{2,n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m,1} & a_{m,2} & \cdots & a_{m,n} \end{bmatrix}$	m by n matrix, first row a sub one one, a sub one two, dot dot dot, a sub one n , second row a sub two one, a sub two two, dot dot dot, a sub two n dot dot dot last row a sub m one, a sub m two, dot dot dot a sub m n
$a_{i,j}$	Lowercase (small) a sub i comma j
a_{ij+1}	Lowercase a double subscript i comma j plus one
a_{ij-1}	Lowercase a double subscript i comma j minus one

range中使用min、max

02659: Bomb Game

matrices, <http://cs101.openjudge.cn/practice/02659/>

```

1  ...
2  for (R, S, P, T) in bombs:
3      for i in range(max(0, R - (P - 1) // 2), min(A, R + (P + 1) // 2)):
4          for j in range(max(0, S - (P - 1) // 2), min(B, S + (P + 1) //
5  2)):
6      ...

```

04133: 垃圾炸弹

matrices, <http://cs101.openjudge.cn/practice/04133/>

```

1  for i in range(max(x-d, 0), min(x+d+1, 1025)):
2      for j in range(max(y-d, 0), min(y+d+1, 1025)):

```

(2) 字符串 (string)

字符串是由字符（字母、数字、符号等）组成的有序集合。

字符串与列表类似，也支持序列操作（如索引、切片、拼接等），但字符串不可变。

常见方法示例（表1-4）：

表1-4 Python字符串常用方法

Method Name	Use	Explanation
<code>center</code>	<code>astring.center(w)</code>	返回宽度为 <code>w</code> 的居中字符串
<code>count</code>	<code>astring.count(item)</code>	返回子串出现次数
<code>lower</code>	<code>astring.lower()</code>	转换为小写
<code>find</code>	<code>astring.find(item)</code>	返回首次出现子串的下标，若不存在则返回 -1
<code>split</code>	<code>astring.split(schar)</code>	按分隔符拆分为字符串列表

例如：

```
1 >>> "CS101".lower()
2 'cs101'
3 >>> "a,b,c".split(",")
4 ['a', 'b', 'c']
```

(3) 元组 (tuple)

元组与列表类似，也是有序集合，但与字符串一样不可变。
语法形式为括号 `()` 内一组逗号分隔的值：

```
1 >>> t = (1, "data", 3.14)
```

元组可用于需要不可修改序列的场景，例如作为字典的键。

(4) 集合 (set)

集合是零个或多个不可变对象的无序集合，不允许重复元素。
语法形式为花括号 `{}`，或使用 `set()` 构造函数：

```
1 >>> s = {1, 2, 3}
2 >>> emptySet = set()
```

集合支持数学运算（表1-5）：

表1-5 Python集合的基本运算

Operation Name	Operator	Explanation
membership	<code>in</code>	判断元素是否在集合中
union	<code> </code>	合并
intersection	<code>&</code>	交集
difference	<code>-</code>	差集
subset	<code><=</code>	判断是否为子集

集合的方法（表1-6）：

表1-6 Python集合常用方法

Method Name	Use	Explanation
union	aset.union(otherset)	返回并集
intersection	aset.intersection(otherset)	返回交集
difference	aset.difference(otherset)	返回差集
issubset	aset.issubset(otherset)	判断是否为子集
add	aset.add(item)	添加元素
remove	aset.remove(item)	移除元素（若不存在则报错）
pop	aset.pop()	随机移除并返回一个元素
clear	aset.clear()	清空集合

(5) 字典 (dict)

字典是由键-值对组成的无序集合。语法形式为花括号 {} 内一系列 key:value：

```
1 >>> student = {"name": "Alice", "id": 1001}
```

键必须是不可变对象（如字符串、整数、元组），值则没有限制。

常见操作（表1-7）：

表1-7 Python字典的基本操作

Operator	Use	Explanation
[]	myDict[k]	通过键访问值，若不存在则报错
in	key in adict	判断键是否存在
del	del adict[key]	删除指定键

常见方法（表1-8）：

表1-8 Python字典常用方法

Method Name	Use	Explanation
keys	adict.keys()	返回所有键（dict_keys对象）
values	adict.values()	返回所有值（dict_values对象）
items	adict.items()	返回所有键-值对（dict_items对象）
get	adict.get(k)	获取键对应的值，不存在时返回 None
get	adict.get(k,alt)	获取键对应的值，不存在时返回指定默认值 alt

例如：

```
1 >>> student.get("name")
2 'Alice'
3 >>> student.get("age", 18)
4 18
```

1.2 基本语法

1.2.1 输入与输出

程序通常需要与用户交互，以便获取输入或输出结果。多数应用使用图形化对话框来收集数据，但在入门阶段，Python 提供了更简洁的方式。

输入：

Python 内置函数 `input` 用于从用户处获取输入。它接受一个字符串作为参数，该字符串常被称为**提示字符串**（prompt），用于提示用户输入内容。

```
1 aName = input("Please enter your name: ")
```

无论用户输入什么，`input` 都会返回一个字符串。若需要使用其他类型（如整数或浮点数），必须显式进行类型转换：

```
1 age = int(input("Please enter your age: "))
```

输出：

`print` 函数用于向屏幕输出信息。默认情况下，参数之间以空格分隔，末尾带有换行符。可以通过 `sep` 参数更改分隔符，通过 `end` 参数修改结尾字符：

```
1 print("Hello", "World", sep="-", end="!")
2 # 输出: Hello-World!
```

当需要更精确地控制输出格式时，可以使用**格式化字符串**。格式化字符串将固定文本与占位符结合，后续由变量填充：

```
1 name = "Alice"
2 age = 20
3 print(f"{name} is {age} years old.")
```

04140: 方程求解

牛顿迭代法，<http://cs101.openjudge.cn/practice/04140/>

```
1 ...
2 print(f"{root2:.9f}")
```

read 一次性读入, 配合 iter, next

有的题目输入数据本来是给C++设计的, 如果用Python需要一次性都读入。

04093: 倒排索引查询

data structures, <http://cs101.openjudge.cn/practice/04093/>

...

样例输入

```
1 3
2 3 1 2 3
3 1 2
4 1 3
5 3
6 1 1 1
7 1 -1 0
8 1 -1 -1
```

...

```
1 import sys
2
3
4 def main():
5     data = sys.stdin.read().split()
6     it = iter(data)
7
8     # 读入倒排索引的词数
9     N = int(next(it))
10    inverted = []
11    for _ in range(N):
12        # 每个词的出现文档数
13        count = int(next(it))
14        docs = set()
15        for _ in range(count):
16            docs.add(int(next(it)))
17        inverted.append(docs)
18
19    # 读入查询数目
20    M = int(next(it))
21    output_lines = []
22    for _ in range(M):
23        # 每个查询包含 N 个数字
24        query = [int(next(it)) for _ in range(N)]
25        candidate = None
26        ...
27        # 输出结果
28        if candidate:
29            result_line = " ".join(map(str, sorted(candidate)))
30            output_lines.append(result_line)
```



```

31         else:
32             output_lines.append("NOT FOUND")
33
34     sys.stdout.write("\n".join(output_lines))
35
36
37 if __name__ == '__main__':
38     main()

```

M12029:水淹七军

bfs, dfs, <http://cs101.openjudge.cn/pctbook/M12029/>

```

1  # 读取并处理输入
2  data = sys.stdin.read().split()
3  k = int(data[0])
4  id = 1
5  ans = []
6  ...

```

09201: Freda的越野跑

<http://cs101.openjudge.cn/practice/09201/>

...

样例输入

```

1  5
2  1 3 10 8 5

```

...

```

1  import sys
2  ...
3  n = int(sys.stdin.readline())
4  a = list(map(int, sys.stdin.readline().split()))

```

1.2.2 控制结构

算法的执行离不开迭代和分支两类控制结构。Python 提供了多种方式来实现。

迭代：

- `while` 语句在条件为真时重复执行：

```
1 while count < 5:
2     print(count)
3     count += 1
```

- `for` 语句更为常用，能够直接遍历序列或其他可迭代对象：

```
1 for item in [1, 2, 3]:
2     print(item)
```

分支：

`if` 语句用于条件判断。可与 `else` 和 `elif` 组合，实现多分支逻辑：

```
1 if x > 0:
2     print("Positive")
3 elif x == 0:
4     print("Zero")
5 else:
6     print("Negative")
```

与多数语言类似，Python 允许嵌套分支，但推荐使用 `elif` 来减少层次，提高可读性。

列表解析式：

除了常规循环，Python 提供了列表解析式（list comprehension），用于基于条件与处理规则快速构建列表：

```
1 squares = [x**2 for x in range(10) if x % 2 == 0]
```

1.2.3 异常处理

在编程过程中，错误主要分为两类：

1. **语法错误**：编写语句或表达式时出错，解释器无法执行。
2. **逻辑错误**：程序能够运行，但结果错误。例如除以零、访问越界，这些通常会引发运行时错误，也称为异常。

Python 提供了机制来处理异常，而不是简单地终止程序。可以使用 `try-except` 语句捕获并处理异常：

```
1 try:
2     result = 10 / 0
3 except ZeroDivisionError:
4     print("Division by zero is not allowed.")
```

此外，程序员还可以在特定情况下主动抛出异常，以便提醒调用者：

```
1 | raise ValueError("Invalid input!")
```

处理不定行输入的常用方法

在 Python 中，处理多行输入有以下几种常见方式：

使用 try...except 捕获输入结束（如 EOFError）

利用 sys.stdin 逐行读取

通过 sys.stdin.read() 一次性读取所有输入



【高中与大学的计算机基础课程衔接】

+ 新建对话

历史对话记录

- Python多行输入
- 函数内外变量作用域
- 函数内外变量作用域
- T25353排队超时
- 学习课程方法
- Typora与Obs
- 编程环境配置
- Python环境配置
- 编程核心与LeetC
- 知识库查询
- 知识库概述

学习技巧

- 课程知识库
- 课程广场
- 闫宏飞

DeepSeek-V3 Python多行输入 对话记录 闫宏飞

处理不定行输入的常用方法 在 Python 中，处理多行输入有以下几种常见方式：使用 try...except 捕获输入结束（如 EOFError） 利用 sys.stdin 逐行读取 通过 sys.stdin.read() 一次性读取所有输入

assistant DeepSeek-V3

在 Python 中，处理不定行输入（即不知道具体行数的输入）有以下几种常用方法：

使用 try-except 捕获 EOFError

```
while True:
    try:
        line = input()
        # 处理每一行的逻辑
    except EOFError:
        break
```

这种方法适用于手动输入时使用 Ctrl+D (Unix/Linux) 或 Ctrl+Z+Enter (Windows) 结束输入的场景[来源 1][来源 4][来源 11]。

利用 sys.stdin 逐行读取

```
import sys
for line in sys.stdin:
    # 处理每一行的逻辑（包含换行符）
    line = line.strip() # 通常需要去除首尾空白
```

通过 sys.stdin.read() 一次性读取所有输入

```
import sys
data = sys.stdin.read() # 整个输入作为字符串
lines = data.splitlines() # 按行分割
for line in lines:
    # 处理每一行的逻辑
```

这种方法特别适合处理大规模输入，效率较高[来源 2][来源 8]。

列表存储法

```
lines = []
while True:
    try:
        lines.append(input())
    except EOFError:
        break
# 后续处理lines列表
```

这种方法将所有输入存储在列表中便于后续处理[来源 4]。

注意事项：

Windows 和 Unix/Linux 系统的EOF输入方式不同

字符串处理时注意换行符和首尾空格

对于算法竞赛等场景，推荐使用sys.stdin.read()方法提高IO效率[来源 2][来源 8][来源 16]

参考来源：

2020fall_cs101.ope... 来源1 来源2 来源3 来源4 来源6 来源9

Book_my_flight_20... 来源5 来源7 2020fall_Codeforce... 来源8 来源16 来源18

20241203_knowled... 来源10 算法笔记:胡凡.pdf 来源11 来源17 20241112_Segment... 来源12

2024fall_LeetCode... 来源13 来源15 Python编程：从入门... 来源14

开始新对话

使用Shift+Enter/Ctrl+Enter换行，Enter发送消息

知识库 3/3

提供的信息仅供参考，请仔细甄别核实

版权所有 2025 © 北京大学计算中心

04015: 邮箱验证

strings, <http://cs101.openjudge.cn/practice/04015>

这题目输入没有明确结束，需要套在try ... except里面。测试时候，需要模拟输入结束，看你是window还是mac。If the user hits EOF (*nix: Ctrl-D, Windows: Ctrl-Z+Return), raise EOFError.

No. 27 / 55

```
1 while True:
2     try:
3         s = input()
4     except EOFError:
5         break
6
7     ...
8     print('NO' if q== -1 else 'YES')
```

1.2.4 定义函数

函数是一种过程抽象工具，可以将复杂的计算隐藏在简洁的调用中。
在 Python 中，函数定义包含：

- 函数名
- 参数列表（可以为空）
- 函数体
- （可选）返回值

示例：

```
1 def square(x):
2     return x * x
3
4 print(square(5)) # 输出 25
```

通过函数，程序员可以将问题分解为更小的部分，提高代码的可读性与复用性。

19942: 二维矩阵上的卷积运算

matrices, <http://cs101.openjudge.cn/practice/19942/>

思路：

定义函数真的太方便啦！

18161:矩阵运算(先乘再加)

1. 代码实现 & 解题思路

```
def juzhen(a, b):
    l = []
    for i in range(a):
        l.append(list(map(int, input().split())))
    return l

a, b = map(int, input().split())
l1 = juzhen(a, b)
a, b = map(int, input().split())
l2 = juzhen(a, b)
a, b = map(int, input().split())
l3 = juzhen(a, b)

def cheng(A, B):
    row_A, col_A = len(A), len(A[0])
    row_B, col_B = len(B), len(B[0])
    result = [[0] * col_B for _ in range(row_A)]
    for i in range(row_A):
        for j in range(col_B):
            for k in range(col_A):
                result[i][j] += A[i][k] * B[k][j]
    return result

def jia(A, B):
    row, col = len(A), len(A[0])
    result = [[A[i][j] + B[i][j] for j in range(col)] for i in range(row)]
    return result

if len(l1[0]) != len(l2):
    print("Error!")
elif len(l1) != len(l3) or len(l2[0]) != len(l3[0]):
    print("Error!")
else:
    AB = cheng(l1, l2)
    result = jia(AB, l3)
    for row in result:
        print(*row)
```

解题思路：为了方便，把所有的过程（乘，加，建矩阵）设置成函数了（def juzhen, def cheng, def jia）新获得的知识：print(*row)真方便！

1.3 面向对象编程

Python 是一门支持面向对象编程（Object-Oriented Programming, OOP）的语言。到目前为止，我们已经使用了一些内置类来展示数据结构与控制流程的基本用法。面向对象编程最强大的特性之一，是允许程序员（即问题求解者）定义全新的类，以精确地对现实世界或抽象问题中的数据建模。

此前，我们通过**抽象数据类型**（Abstract Data Type, ADT）从逻辑上描述了数据对象的状态及其行为。通过构建实现这些抽象数据类型的类，我们既能利用抽象思维，又能为程序中实际使用这些抽象提供具体的实现细节。每当需要实现一个抽象数据类型时，创建一个新的类便成为自然的选择。

Q: ADT是Class还高级的抽象？

简单来说：是的，**ADT（抽象数据类型）比 Class（类）更高级、更抽象。**

可以从以下几个层面来理解它们之间的关系：

1. 定义上的区别

概念	定义
ADT (Abstract Data Type)	是一种逻辑上的数学模型，它只关心“能做什么”——即定义了一组操作（如 <code>push</code> , <code>pop</code> 对于栈），以及这些操作的数学性质和行为规范，但完全不涉及“如何实现”。
Class (类)	是一种具体的编程语言构造，它描述了“怎么做”——即用代码实现 ADT 所要求的数据结构和方法，并包含具体的状态（字段/属性）和行为（方法）。

类比：

- ADT 像是一份**产品需求说明书**：“这个东西必须能存数据、取数据、返回大小。”
- Class 是根据这份说明书写**的工程图纸或实际产品**：“我用数组/链表实现了这个功能。”

2. 抽象层次对比

1	最高层抽象
2	
3	— 抽象数据类型 (ADT)
4	└ 只定义接口与语义 (What)
5	例如：栈 (Stack)、队列 (Queue)、集合 (Set)、分数 (Fraction)
6	
7	— 数据结构 (Data Structure)
8	└ 实现 ADT 的一种方式 (How)
9	例如：用数组实现栈、用链表实现队列
10	
11	— 类 (Class)
12	└ 在面向对象语言中的具体编码实现
13	例如：Python 中的 <code>`class Stack:`</code>

所以：

- **ADT** 是思想层面的抽象；
- **Class** 是实现层面的具体化。

3. 举例说明：栈 (Stack)

层次	内容
ADT: 栈	定义两个核心操作： - <code>push(item)</code>：将元素压入栈顶 - <code>pop()</code>：移除并返回栈顶元素 行为遵循 LIFO（后进先出）原则。
数据结构实现	可以用数组或链表来实现这个“后进先出”的行为。
Class 实现（Python 示例）	<pre> class Stack: def init(self): self.items = [] def push(self, item): self.items.append(item) def pop(self): return self.items.pop() </pre>

⚠：上面的 `class Stack` 是对“栈”这一 ADT 的一个具体实现。也可以用链表重新实现一次 `Stack` 类，只要行为符合 ADT 规范，就是合法的。

4. 为什么说 ADT 更“高级”？

- **独立于语言**：ADT 可以在任何编程范式中存在（函数式、过程式、面向对象），而 Class 是 OOP 特有的。
- **屏蔽细节**：ADT 隐藏了所有实现细节，使用者只需知道“接口”，不必关心内部是数组还是链表。
- **可替换性**：同一个 ADT 可以有多种 Class 实现。比如 `List` 这个 ADT，可以用 `ArrayList` 或 `LinkedList` 来实现。

5. 之后要实现的例子：Fraction

在原文中：

- **ADT 层面**：想要一个“分数”类型，支持加减乘除、比较、最简形式输出等。
- **Class 层面**：用 Python 的 `class Fraction` 来实现这个想法，使用两个整数 `num` 和 `den` 存储状态，并重写 `__add__`, `__eq__` 等方法。

所以，“Fraction”作为一个抽象数据类型，先于 `Fraction` 类而存在。类只是它的实现载体。

总结

维度	ADT	Class
抽象级别	高（概念层）	低（实现层）
关注点	“做什么”（What）	“怎么做”（How）
是否依赖语言	否	是（依赖 OOP 语言）
是否可执行	否（只是一个规范）	是（可运行的代码）
多态性	行为多态（同一接口不同实现）	支持继承与多态

一句话总结：

ADT 是蓝图，Class 是按照蓝图建造的房子。

一栋房子可以有不同的建造方式（钢筋混凝土 or 木结构），就像一个 ADT 可以有多个 Class 实现。

因此，ADT 确实比 Class 更高级、更抽象。

1.3.1 实现用户定义的类

示例E27653:Fraction类

OOP, <http://cs101.openjudge.cn/pctbook/E27653/>

实现用户定义的类，一个常用的例子是构建实现抽象数据类型Fraction的类。我们已经看到，Python提供了很多数值类。但是在有些时候，需要创建“看上去很像”分数的数据对象。

像 3/5 这样的分数由两部分组成。斜线左侧的值称作分子，可以是任意整数。斜线右侧的值称作分母，可以是任意大于0的整数（负的分母带有负的分母）。尽管可以用浮点数来近似表示分数，但我們在此希望能精确表示分数的值。

Fraction对象的表现应与其他数值类型一样，针对分数进行加、减、乘、除等运算，也能够使用标准的斜线形式来显示分数，比如3/5。此外，所有的分数方法都应该返回结果的最简形式。这样一来，不论进行何种运算，最后的结果都是最简分数。

出于练习考虑，只需要支持一种分数加运算。

输入

空格分割的一行字符串。第一个分数的分子 第一个分数的分母 第二个分数的分子 第二个分数的分母

输出

输出相加的结果。使用标准的斜线形式来显示分数，并且要求是最简分数。

样例输入

```
1 | 1 4 1 2
```

样例输出

```
1 | 3/4
```

提示

OOP

来源

yan, <https://runestone.academy/ns/books/published/pythonds3/Introduction/ObjectOrientedProgrammingin>

为了展示如何定义用户自定义类，以实现一个名为 `Fraction` 的类为例，该类用于表示数学中的分数。虽然 Python 已经提供了多种数值类型（如 `int` 和 `float`），但浮点数存在精度损失的问题，无法精确表示所有分数。因此，我们需要一种方式来精确表示并操作分数。

一个分数由两部分组成：

- **分子** (numerator)：可以是任意整数；
- **分母** (denominator)：必须是大于 0 的正整数（负数通过负的分子表示）。

例如，分数 $\frac{3}{5}$ 的分子为 3，分母为 5。我们的目标是让 `Fraction` 对象的行为与其他数值类型相似：支持加、减、乘、除等运算，能够以标准形式（如 `3/5`）输出，并且所有运算结果都应化简为最简形式。

Q: 用 `fractions.Fraction` 的代码

如果题目允许使用 Python 标准库（通常在实际开发中推荐，但在某些考察“类实现”原理的算法题中可能被禁止），使用 `fractions.Fraction` 会让代码极其简洁，因为它内部已经完美实现了约分、符号处理和算术运算。

以下是使用 `fractions.Fraction` 的解法：

代码实现

```
1  from fractions import Fraction
2  import sys
3
4  def main():
5      # 读取输入行
6      try:
7          line = sys.stdin.read().split()
8          if not line:
9              return
10
11         # 解析四个整数
12         n1 = int(line[0])
13         d1 = int(line[1])
14         n2 = int(line[2])
15         d2 = int(line[3])
16
17         # 创建 Fraction 对象
18         # Fraction 会自动处理约分和符号（分母永远为正）
19         f1 = Fraction(n1, d1)
20         f2 = Fraction(n2, d2)
21
22         # 执行加法
```

```

23     result = f1 + f2
24
25     # 输出结果
26     # Fraction 的默认字符串格式就是 "分子/分母"
27     print(result)
28
29     except Exception:
30         pass
31
32 if __name__ == "__main__":
33     main()

```

为什么这段代码能工作？

1. 自动约分：

`Fraction(1, 4) + Fraction(1, 2)` 内部计算后会自动化简为 `Fraction(3, 4)`，无需手动写 GCD 逻辑。

2. 自动处理符号：

如果你传入 `Fraction(1, -2)`，它会自动存储为 `Fraction(-1, 2)`，完全符合题目“负号在分子”的要求。

3. 默认输出格式：

`print(Fraction(3, 4))` 直接输出字符串 `"3/4"`，不需要额外编写 `__str__` 方法。

4. 运算符重载：

直接使用 `+` 号即可，不需要调用 `.add()` 方法。

⚠ 重要提示：关于提交

- 如果这是一道“数据结构/算法原理”练习题（如题目描述中强调“实现用户定义的类”、“构建实现抽象数据类型Fraction的类”）：
考官的目的是考察你是否理解最大公约数算法和分数运算公式。在这种情况下，使用 `fractions` 模块可能会被判为“作弊”或“未按要求实现”而导致 **Wrong Answer** 或 **Runtime Error**（如果系统禁用了该模块）。
- 如果这是一道“应用开发”或“脚本编写”题：
那么使用标准库是最佳实践，代码更健壮且不易出错。

建议：如果是为了通过特定的算法课程作业，请优先使用我上一条回答中的手动实现版；如果是为了解决实际问题或比赛不限制库，请使用这个标准库版。

我刚去掉了一组测试数据。因为发现：`fractions.Fraction` 更严谨，发现了一个输入数据 0 1 -20 2，应该输出 -10。之前这组数据输出-10/1。

定义类的基本结构

在 Python 中，定义一个新类的基本语法如下：

```

1 class ClassName:
2     # 方法定义
3     pass

```

说明：在 Python 中，所有类默认继承自 `object` 类，它是所有类的根类。如果未显式指定父类，Python 会自动将其设为 `object` 的子类。

类中的每个方法都必须将 `self` 作为第一个参数。`self` 是一个指向当前实例对象的引用，由 Python 在调用方法时自动传入。开发者在调用方法时无需手动传递该参数。这种机制使得不同实例可以调用相同的方法，而方法内部能正确访问对应实例的数据。

构造方法： `__init__`

每个类通常都需要一个构造方法（constructor），用于定义对象的初始化过程。在 Python 中，构造方法的名称固定为 `__init__`（注意前后各有两个下划线）。

以下是 `Fraction` 类的构造方法实现：

```
1 def __init__(self, top, bottom):
2     self.num = top      # 分子
3     self.den = bottom   # 分母
```

该方法接受三个参数：`self`、`top`（分子）和 `bottom`（分母）。其中，`self.num` 和 `self.den` 是对象的实例变量（也称属性），用于保存对象的状态信息。

创建 `Fraction` 实例时，只需调用类名并传入相应的参数即可（无需直接调用 `__init__`）：

```
1 myfraction = Fraction(3, 5)
```

这行代码创建了一个值为 $\frac{3}{5}$ 的 `Fraction` 对象，其内部状态如图 2-1 所示。

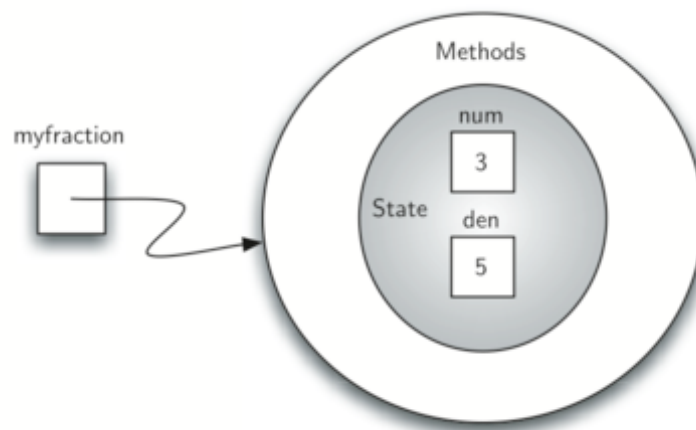


图2-1 Fraction类的一个实例

自定义字符串表示： `__str__` 方法

如果我们尝试打印一个 `Fraction` 对象：

```
1 myf = Fraction(3, 5)
2 print(myf)
```

输出可能是类似：

```
1 <__main__.Fraction object at 0x...>
```

这是 Python 默认的字符串表示，仅显示对象的内存地址，显然不符合我们的需求。我们希望

`print(myf)` 能输出 `3/5`。

为此，我们需要重写类中的 `__str__` 方法。`__str__` 是 Python 内置的标准方法，用于将对象转换为可读的字符串形式。其默认实现仅返回对象的地址信息，因此我们需要提供更合适的实现：

```
1 def __str__(self):
2     return f"{self.num}/{self.den}"
```

现在再次测试：

```
1 myf = Fraction(3, 5)
2 print(myf) # 输出: 3/5
3 print("I ate", myf, "of the pizza") # 输出: I ate 3/5 of the pizza
4 str(myf) # 返回: '3/5'
```

提示：`__str__` 方法被 `print()` 函数、字符串格式化操作以及 `str()` 函数自动调用。

虽然也可以定义一个 `show()` 方法来打印分数：

```
1 def show(self):
2     print(self.num, "/", self.den)
```

但这并不通用，因为 `print()` 不会自动调用它。相比之下，`__str__` 是标准接口，能无缝集成到 Python 的输出系统中。

实现加法运算：`__add__` 方法

目前，如果尝试将两个 `Fraction` 对象相加：

```
1 f1 = Fraction(1, 4)
2 f2 = Fraction(1, 2)
3 f1 + f2
```

会抛出异常：

```
1 TypeError: unsupported operand type(s) for +: 'Fraction' and 'Fraction'
```

这是因为 Python 不知道如何对 `Fraction` 类型执行加法操作。要解决这个问题，我们需要重写 `__add__` 方法。

`__add__` 方法接收两个参数：`self` 和 `other`（表示另一个操作数）。它应返回一个新的 `Fraction` 对象作为结果。

根据分数加法规则：

$$\frac{a}{b} + \frac{c}{d} = \frac{ad + bc}{bd} \quad (1)$$

我们可实现如下：

```

1 def __add__(self, otherfraction):
2     newnum = self.num * otherfraction.den + self.den * otherfraction.num
3     newden = self.den * otherfraction.den
4     return Fraction(newnum, newden)

```

测试：

```

1 f1 = Fraction(1, 4)
2 f2 = Fraction(1, 2)
3 f3 = f1 + f2
4 print(f3) # 输出: 6/8

```

虽然结果正确，但 `6/8` 并非最简形式。理想结果应为 `3/4`。为此，我们需要在返回前对结果进行约分。

辅助函数：求最大公因数（GCD）

为了化简分数，我们需要计算分子和分母的最大公因数（Greatest Common Divisor, GCD），然后将两者同时除以 GCD。

最经典的算法是欧几里得算法（Euclidean Algorithm），其原理如下：

- 若 $m \bmod n = 0$ ，则 $\text{gcd}(m, n) = n$ ；
- 否则， $\text{gcd}(m, n) = \text{gcd}(n, m \bmod n)$ 。

迭代实现如下：

```

1 def gcd(m, n):
2     while m % n != 0:
3         m, n = n, m % n
4     return n

```

⚠：该实现要求分母为正数，这与我们之前对 `Fraction` 的定义一致（负数由负分子表示）。

我们可以使用 Python 标准库验证结果：

```

1 import math
2 print(math.gcd(20, 10)) # 输出: 10

```

改进 `__add__` 方法以返回最简分数

结合 `gcd` 函数，我们改进 `__add__` 方法：

```

1 def __add__(self, otherfraction):
2     newnum = self.num * otherfraction.den + self.den * otherfraction.num
3     newden = self.den * otherfraction.den
4     common = gcd(newnum, newden)
5     return Fraction(newnum // common, newden // common)

```

再次测试：

```

1 f1 = Fraction(1, 4)
2 f2 = Fraction(1, 2)
3 f3 = f1 + f2
4 print(f3) # 输出: 3/4

```

结果已是最简形式。

实现相等性比较：`__eq__` 方法

默认情况下，Python 使用“浅相等”（shallow equality）判断对象是否相等，即仅当两个变量引用同一个对象时，`==` 才返回 `True`。例如：

```

1 f1 = Fraction(1, 2)
2 f2 = Fraction(1, 2)
3 print(f1 == f2) # False (即使数值相同)

```

但我们希望实现“深相等”（deep equality），即根据对象的实际值（分子与分母）来判断是否相等。

为此，需重写 `__eq__` 方法：

```

1 def __eq__(self, other):
2     firstnum = self.num * other.den
3     secondnum = other.num * self.den
4     return firstnum == secondnum

```

原理：交叉相乘，比较 $a \cdot d$ 与 $b \cdot c$ 是否相等。

测试：

```

1 f1 = Fraction(1, 2)
2 f2 = Fraction(2, 4)
3 print(f1 == f2) # True

```

这样，即使分数形式不同，只要值相同，就被视为相等。

其他关系运算符（如 `<`，`<=`，`!=` 等）也可通过类似方式重写，例如 `__le__` 表示小于等于。

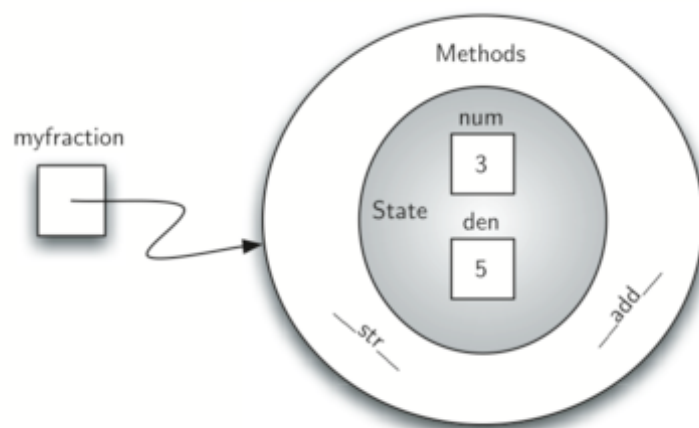


图2-2 包含两个方法的Fraction实例

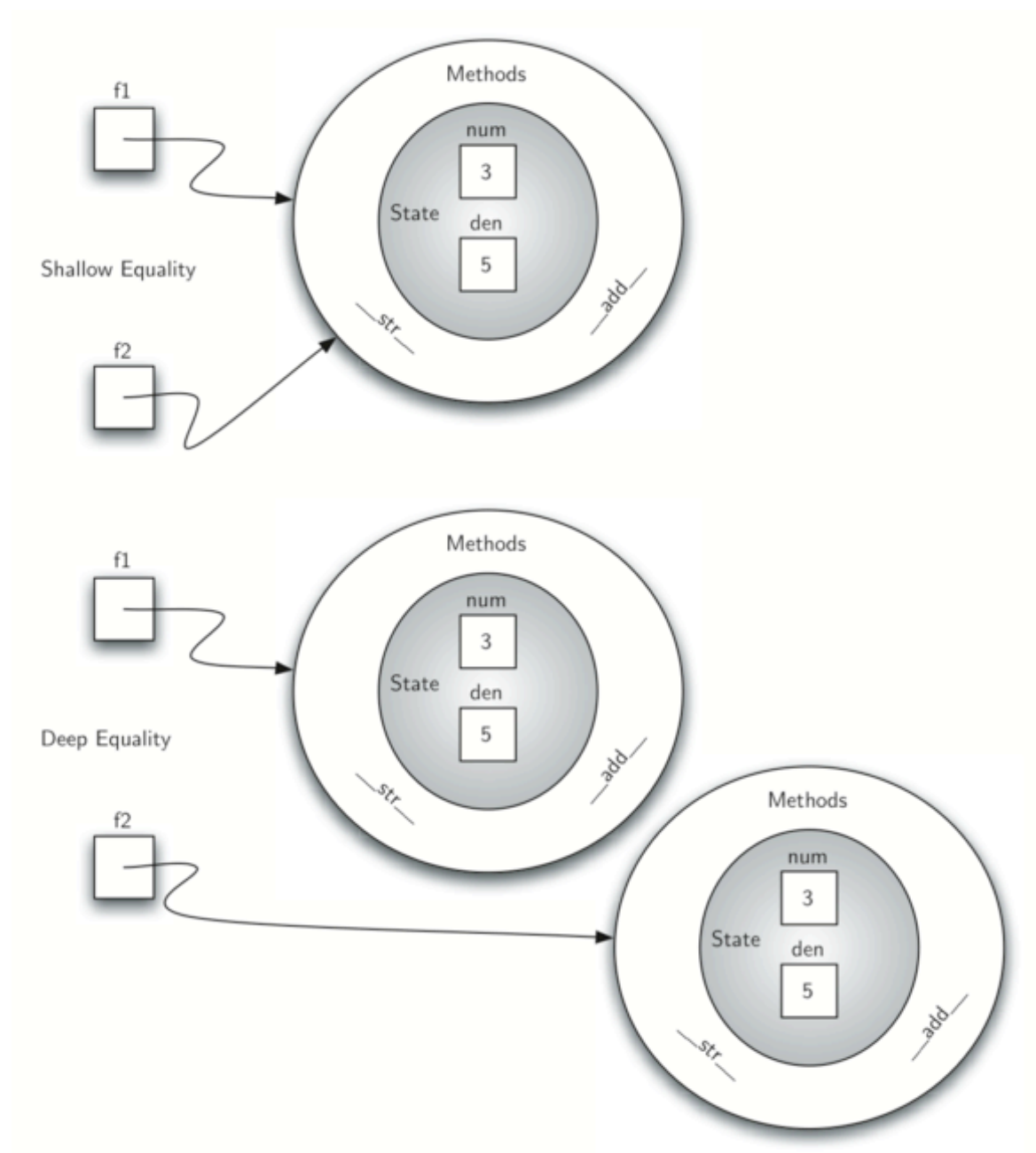


图2-3 浅相等与深相等

完整实现代码

以下是 `Fraction` 类的完整实现：

```
1 def gcd(m,n):
2     while m%n != 0:
3         oldm = m
4         oldn = n
5
```



```

6         m = oldn
7         n = oldm%oldn
8     return n
9
10 class Fraction:
11     def __init__(self, top, bottom):
12         self.num = top
13         self.den = bottom
14
15     def __str__(self):
16         return str(self.num)+"/"+str(self.den)
17
18     def show(self):
19         print(self.num, "/", self.den)
20
21     def __add__(self, otherfraction):
22         newnum = self.num*otherfraction.den + \
23                 self.den*otherfraction.num
24         newden = self.den * otherfraction.den
25         common = gcd(newnum, newden)
26         return Fraction(newnum//common, newden//common)
27
28     def __eq__(self, other):
29         firstnum = self.num * other.den
30         secondnum = other.num * self.den
31
32         return firstnum == secondnum
33
34 # 测试代码
35 x = Fraction(1, 2)
36 y = Fraction(2, 3)
37 print(x + y)      # 输出: 7/6
38 print(x == y)    # 输出: False

```

小结

本节通过构建 `Fraction` 类，系统地展示了面向对象编程的核心概念：

- 使用 `class` 定义新类型；
- 利用 `__init__` 初始化对象状态；
- 通过 `__str__` 自定义对象的字符串表示；
- 重写 `__add__` 和 `__eq__` 等特殊方法，使自定义类具备与内置类型相似的行为；
- 引入辅助函数（如 `gcd`）提升功能完整性。

`Fraction` 类的成功实现，体现了面向对象技术在封装数据与行为、提升代码可读性与可维护性方面的强大优势。其余算术运算（减、乘、除）和比较运算符的实现，可作为练习进一步巩固所学知识。

Q: “实现 Python 数据模型（Data Model）”，意思就是：通过实现特定的魔术方法（双下划线方法），让你的自定义类能够像 Python 内置类型（如 `int`, `list`, `dict`）一样工作。

下面是一份完整的、针对 OOP 练习优化的代码，重点展示了三个核心点：`__init__`（构造与清洗数据）、`__add__`（运算符重载）、`__str__`（自定义显示）。

```

1  import math
2
3  class Fraction:
4  def __init__(self, numerator, denominator):
5      """
6      构造函数 (__init__)
7      职责:
8          1. 接收原始数据。
9          2. 数据清洗: 处理分母为负的情况 (将负号移至分子)。
10         3. 数据标准化: 自动约分为最简分数。
11         """
12         if denominator == 0:
13             raise ValueError("分母不能为0")
14
15         # 1. 符号规范化: 保证分母永远为正
16         if denominator < 0:
17             numerator = -numerator
18             denominator = -denominator
19
20         # 2. 自动约分: 利用最大公约数 (GCD)
21         common = math.gcd(numerator, denominator)
22         self.numerator = numerator // common
23         self.denominator = denominator // common
24
25     def __add__(self, other):
26         """
27         运算符重载 (__add__)
28         职责:
29             1. 定义 '+' 号的行为。
30             2. 实现分数加法公式:  $a/b + c/d = (ad + bc) / bd$ 
31             3. 返回一个新的 Fraction 对象 (触发新的 __init__ 进行自动约分)
32         """
33         if not isinstance(other, Fraction):
34             # 可选: 支持 Fraction + int 的情况, 这里暂只处理 Fraction +
Fraction
35             return NotImplemented
36
37             new_num = self.numerator * other.denominator + other.numerator
* self.denominator
38             new_den = self.denominator * other.denominator
39
40             # 返回新实例, 构造函数会自动处理约分和符号
41             return Fraction(new_num, new_den)
42
43     def __str__(self):
44         """
45         自定义字符串表示 (__str__)
46         职责:
47             1. 定义 print() 或 str() 时的输出格式。
48             2. 特殊情况处理: 如果分母为1, 直接输出整数 (如 -10), 否则输出 a/b。
49         """
50         if self.denominator == 1:
51             return str(self.numerator)

```

```

52         else:
53             return f"{self.numerator}/{self.denominator}"
54
55     def __repr__(self):
56         """
57         (额外补充) 开发者视角的字符串表示
58         用于在交互式命令行直接查看对象详情
59         """
60         return f"Fraction({self.numerator}, {self.denominator})"
61
62     # --- 主程序逻辑 ---
63     def main():
64         # 模拟输入读取
65         try:
66             # 假设输入来自 stdin, 这里用 input().split() 模拟
67             # 实际提交时请确保处理可能的多行或空格问题
68             data = input().split()
69             if len(data) != 4:
70                 return
71
72             n1, d1, n2, d2 = map(int, data)
73
74             # 1. 实例化对象 (调用 __init__)
75             f1 = Fraction(n1, d1)
76             f2 = Fraction(n2, d2)
77
78             # 2. 执行加法 (调用 __add__)
79             # 写法 f1 + f2 等价于 f1.__add__(f2)
80             result = f1 + f2
81
82             # 3. 输出结果 (调用 __str__)
83             # 写法 print(result) 等价于 print(str(result))
84             print(result)
85
86         except Exception as e:
87             # 调试时可打开查看错误
88             # print(f"Error: {e}")
89             pass
90
91     if __name__ == "__main__":
92         main()

```

关键点解析 (对应你的练习目标)

1. 构造函数 `__init__`: 数据的守门员

- 作用: 不仅仅是赋值, 更是数据不变量 (Invariant) 的维护者。
- 练习点: 你在 `__init__` 里做了两件事:
 1. 符号归一化: 确保 `1/-2` 变成 `-1/2`。这样后续所有运算都不用再判断分母的正负了。
 2. 即时约分: 确保对象一旦创建, 内部状态就是最简的 (`2/4` 变成 `1/2`)。这使得后续的 `__add__` 逻辑可以假设输入已经是简化的 (虽然公式本身不依赖此假设, 但能防止数字无限膨胀)。

2. 运算符重载 `__add__`：让自定义类型像原生类型一样工作

- 作用：拦截 `+` 操作符。
- 练习点：
 - 如果不写这个方法，`f1 + f2` 会报错 `TypeError`。
- 写了这个方法，Python 解释器看到 `+` 就会去找左操作数的 `__add__` 方法。
- 返回值：注意返回的是 `Fraction(...)` 新对象，而不是修改 `self`。这符合 Python 数值类型（如 `int`, `float`）不可变 (Immutable) 的特性。

3. 字符串表示 `__str__`：控制用户看到的界面

- 作用：决定 `print(obj)` 输出什么。
- 练习点：
 - 你提到的 `0 1 -20 2 -> -10` 的案例，就是在这里解决的。
 - 逻辑：`if self.denominator == 1: return str(self.numerator)`。
 - 如果没有这个判断，输出会是 `-10/1`，虽然数学上对，但不符合人类阅读习惯，也可能不符合 OJ 的判题字符串匹配。

总结

这个练习非常经典，它涵盖了 OOP 的三个重要概念：

1. **封装 (Encapsulation)**：通过 `__init__` 把复杂的约分逻辑藏在对象内部，外部使用者只需关心 `Fraction(1, 2)`，不用管它是怎么简化的。
2. **多态 (Polymorphism)**：通过 `__add__`，让 `Fraction` 也能使用 `+` 号，就像 `int` 和 `float` 一样。
3. **抽象 (Abstraction)**：通过 `__str__`，向用户暴露简洁的数学形式，隐藏内部的 `numerator` 和 `denominator` 属性细节。

这就是为什么 Python 的 `fractions` 模块如此强大且“严谨”，因为它完美地实现了这套数据模型。你自己动手实现一遍，对理解 Python 的对象机制会有极大的帮助！

示例M27300: 模型整理

sortings, AI, <http://cs101.openjudge.cn/pctbook/M27300/>

深度学习模型（尤其是大模型）是近两年计算机学术和业界热门的研究方向。每个模型可以用“模型名称-参数量”命名，其中参数量的单位会使用两种：M，即百万；B，即十亿。同一个模型通常有多个不同参数的版本。

例如，Bert-110M, Bert-340M 分别代表参数量为 1.1 亿和 3.4 亿的 Bert 模型，GPT3-350M, GPT3-1.3B 和 GPT3-175B 分别代表参数量为 3.5 亿，13 亿和 1750 亿的 GPT3 模型。

参数量的数字部分取值在 [1, 1000) 区间（一个 8 亿参数的模型表示为 800M 而非 0.8B，10 亿参数的模型表示为 1B 而非 1000M）。

计算机专业的学生小 A 从网上收集了一份模型的列表，他需要将它们按照名称归类排序，并且同一个模型的参数量从小到大排序，生成“模型名称: 参数量1, 参数量2, ...”的列表。请你帮他写一个程序实现。

输入

第一行为一个正整数 n ($n \leq 1000$)，表示有 n 个待整理的模型。

接下来 n 行，每行一个“模型名称-参数量”的字符串。模型名称是字母和数字的混合。

输出

每行一个“模型名称: 参数量1, 参数量2, ...”的字符串，符号均为英文符号，模型名称按字典序排列，参数量按从小到大排序。

样例输入

```
1 5
2 GPT-1.3B
3 Bert-340M
4 GPT-350M
5 Bert-110M
6 GPT-175B
```

样例输出

```
1 Bert: 110M, 340M
2 GPT: 350M, 1.3B, 175B
```

提示

tags: string, sort

来源

2023fall zyn

定义一个 `ParamSize` 类

```
1 from collections import defaultdict
2
3 class llm:
4
5     def __init__(self, fullname: str) -> None:
6         self.name, self.m = fullname.split('-')
7         self.unit = self.m[-1]
8         self.num = eval(self.m[:-1])
9
10    def __lt__(self, other):
11        if self.unit != other.unit:
12            return self.unit == 'M' # 【关键逻辑】
13        else:
14            return self.num < other.num
15
16    d = defaultdict(list)
17    n = int(input())
18    for i in range(n):
19        l = llm(input())
20        d[l.name].append(l)
21    names = sorted(d.keys())
```

```

22 for i in names:
23     llms = sorted(d[i])
24     print(f'{i}: {" ".join([j.m for j in llms])}')

```

参数量的数字部分取值在 [1, 1000) 区间（一个 8 亿参数的模型表示为 800M 而非 0.8B，10 亿参数的模型表示为 1B 而非 1000M）。所以

```

1     if self.unit!=other.unit:
2         return self.unit=='M'

```

【戴嘉震, 24EECS】定义一个 `ParamSize` 类，继承 `str`，专门用于定义模型大小的排序。

```

1 from collections import defaultdict
2
3 class ParamSize(str):
4     def __lt__(self, other):
5         metri_self = self[-1]
6         metri_other = other[-1]
7         if metri_other == metri_self:
8             num_self = float(self[:-1])
9             num_other = float(other[:-1])
10            return num_self < num_other
11        if metri_self == 'M':
12            return True
13        else:
14            return False
15
16
17 models = defaultdict(list)
18 n = int(input())
19 for _ in range(n):
20     s = input().split('-')
21     models[s[0]].append(ParamSize(s[1]))
22 for key in sorted(models.keys()):
23     print(f'{key}: {" ".join(sorted(models[key]))}')

```

1.3.2 矩阵乘法运算符

示例E18161: 矩阵运算(先乘再加)

matrices, <http://cs101.openjudge.cn/pctbook/E18161/>

现有三个矩阵A, B, C，要求矩阵运算A·B+C并输出结果

矩阵运算介绍：

矩阵乘法运算必须要前一个矩阵的列数与后一个矩阵的行数相同，
如m行n列的矩阵A与n行p列的矩阵B相乘，可以得到m行p列的矩阵C，
矩阵C的每个元素都由A的对应行中的元素与B的对应列中的元素一一相乘并求和得到，
即 $C[i][j] = A[i][0]*B[0][j] + A[i][1]*B[1][j] + \dots + A[i][n-1]*B[n-1][j]$

($C[i][j]$ 表示C矩阵中第i行第j列元素)。

矩阵的加法必须在两个矩阵行列数均相等的情况下进行，
如m行n列的矩阵A与m行n列的矩阵B相加，可以得到m行n列的矩阵C，
矩阵C的每个元素都由A与B对应位置的元素相加得到，
即 $C[i][j] = A[i][j] + B[i][j]$

输入

输入分为三部分，分别是A,B,C三个矩阵的内容。
每一部分的第一行为两个整数，代表矩阵的行数row和列数col
接下来row行，每行有col个整数，代表该矩阵这一行的每个元素

输出

如果可以完成矩阵计算，输出计算结果，与输入格式类似，不需要输出行数和列数信息。
如果不能完成矩阵计算，输出"Error!"

样例输入

```
1 Sample Input1:
2 3 1
3 0
4 1
5 0
6 1 2
7 1 1
8 3 2
9 3 1
10 3 1
11 3 1
12
13 Sample Output1:
14 3 1
15 4 2
16 3 1
```

样例输出

```
1 Sample Input2:
2 1 1
3 0
4 2 1
5 1
6 3
7 1 1
8 9
9
10 Sample Output2:
11 Error!
```

提示

sample1 计算过程

$$\begin{array}{r|l} 0 & 00 \\ 1 \cdot 11 & = 11 \\ 0 & 00 \\ \hline 00 & 31 \\ 11 + 31 & = 42 \\ 00 & 31 \end{array}$$

来源：cs101-2017 期末机考备选 & 2018 Mock Exam 2

思路：矩阵运算，如果没有学过可以百度下矩阵乘法（这是线性代数/高等代数的初步）

```
1  A,B,C = [],[],[]
2
3  a,b = map(int, input().split())
4  for i in range(a):
5      A.append(list(map(int, input().split())))
6
7  c,d = map(int, input().split())
8  for i in range(c):
9      B.append(list(map(int, input().split())))
10
11 e,f = map(int, input().split())
12 for i in range(e):
13     C.append(list(map(int, input().split())))
14
15 if b!=c or a!=e or d!=f:
16     print("Error!")
17 else:
18     D = [[0 for j in range(f)] for i in range(e)]
19     for i in range(e):
20         for j in range(f):
21             for k in range(b):
22                 D[i][j] += A[i][k] * B[k][j]
23                 D[i][j] += C[i][j]
24
25     for i in range(e):
26         print(' '.join([str(j) for j in D[i]]))
```

@ 运算符在 Python 中就是矩阵乘法（从 Python 3.5 开始引入 PEP 465），但它只在支持 `__matmul__` 的对象中有效，比如 `list` 自己封装的矩阵类。

⚠ 标准库里的 `list` 并没有定义 `__matmul__`，所以直接对普通嵌套列表写 `A @ B` 会报错。

办法就是自己封装一个矩阵类，实现 `__matmul__` 和 `__add__` 方法。

代码：

```
1  class Matrix:
2      def __init__(self, data):
3          self.data = data
```



```

4         self.rows = len(data)
5         self.cols = len(data[0]) if self.rows else 0
6
7     def __matmul__(self, other): # 定义 A @ B
8         if self.cols != other.rows:
9             raise ValueError("Matrix dimensions do not match for
multiplication")
10        result = [[0] * other.cols for _ in range(self.rows)]
11        for i in range(self.rows):
12            for j in range(other.cols):
13                for k in range(self.cols):
14                    result[i][j] += self.data[i][k] * other.data[k][j]
15        return Matrix(result)
16
17    def __add__(self, other): # 定义 A + B
18        if self.rows != other.rows or self.cols != other.cols:
19            raise ValueError("Matrix dimensions do not match for addition")
20        result = [
21            [self.data[i][j] + other.data[i][j] for j in range(self.cols)]
22            for i in range(self.rows)
23        ]
24        return Matrix(result)
25
26    def __str__(self): # 打印友好
27        return "\n".join(" ".join(map(str, row)) for row in self.data)
28
29
30    # === 读入 ===
31    def read_matrix():
32        r, c = map(int, input().split())
33        data = [list(map(int, input().split())) for _ in range(r)]
34        return Matrix(data)
35
36
37    A = read_matrix()
38    B = read_matrix()
39    C = read_matrix()
40
41    # === 计算 ===
42    try:
43        D = A @ B + C
44        print(D) # 自动调用 __str__
45    except ValueError:
46        print("Error!")

```

`A @ B` 调用 `__matmul__`，就是矩阵乘法

`(A @ B) + C` 调用 `__add__`，就是矩阵加法

如果维度不合法，抛 `ValueError`，就能捕获并输出 `"Error!"`

1.3.3 继承 (Inheritance)

继承是面向对象编程（OOP）的四大特性之一（封装、抽象、多态、继承），它允许一个类（子类 / 派生类）基于另一个类（父类 / 超类 / 基类）来构建，自动获得父类的数据属性和方法行为。

类比：孩子从父母那里继承基因和习惯，但也可以发展出自己的独特特征。

IS-A 关系

- 如果类 A 是类 B 的子类，则可以说“A 是一个 B”。
- 例如：
 - list 是一个 sequence → list IS-A sequence
 - Car 是一个 Vehicle → Car IS-A Vehicle

这表示子类具备父类的所有基本能力，同时可以扩展或重写功能。

Python 内建集合的继承层次结构，如图2-4。

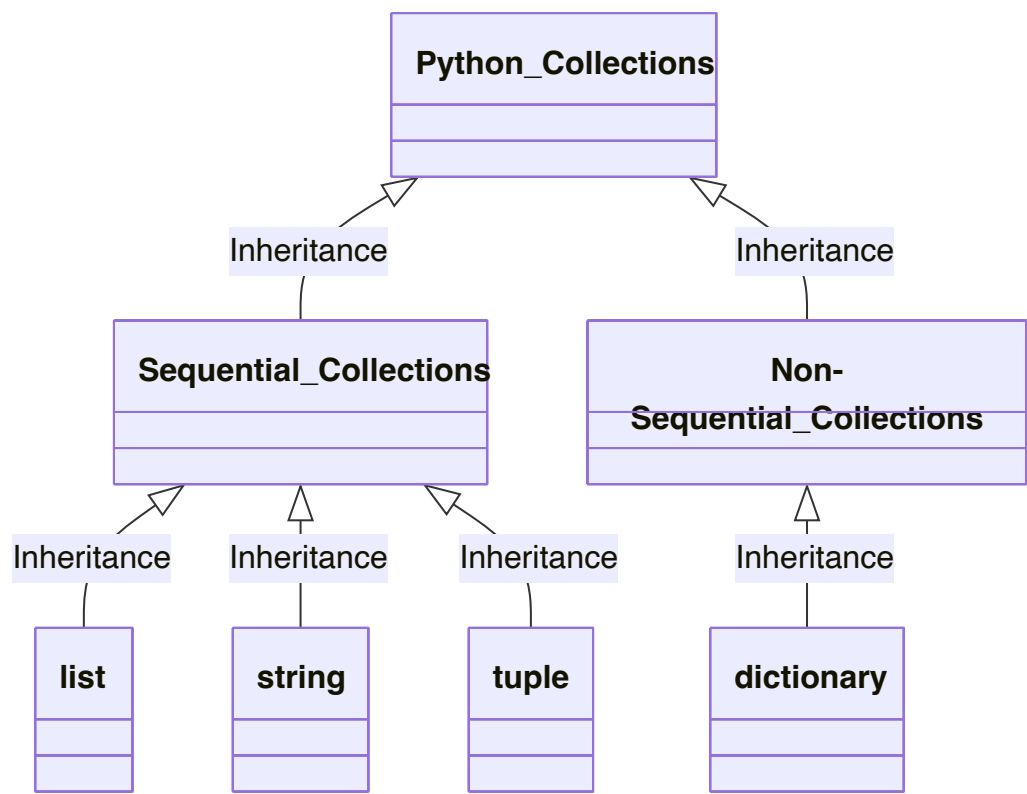


图2-4 Python容器的继承层次结构

分析：

类型	特点	是否可变
<code>list</code>	有序、可变、元素可重复	✅ 可变
<code>tuple</code>	有序、不可变、常用于记录	❌ 不可变
<code>str</code>	有序字符序列	❌ 不可变
<code>dict</code>	无序键值对、通过键访问	✅ 可变

尽管 `string` 和 `tuple` 看似不像是“集合”，但在 Python 中它们都支持索引、切片、遍历等序列操作，因此属于 `Sequential_Collections`。

关键点：
所有这些类型都共享某些通用行为（如长度 `len()`、迭代、成员检测），因为它们共同继承自更高层的抽象结构——这正是继承带来的代码复用优势。

列表、字符串和元组都是有序集合。它们都继承了共同的数据组织和操作。不过，根据数据是否同类以及集合是否可修改，它们彼此又有区别。子类从父类继承共同的特征，但是通过额外的特征彼此区分。

通过将类组织成继承层次结构，面向对象编程语言使以前编写的代码得以扩展到新的应用场景中。此外，这种结构有助于更好地理解各种关系，从而更高效地构建抽象表示。

扩展知识： `namedtuple` —— 更优雅的元组

`namedtuple` 是 Python 的 `collections` 模块中提供了一种数据结构，它扩展了标准元组（`tuple`）的功能。与普通元组不同的是，`namedtuple` 允许你通过名称访问元素，而不仅仅是通过索引，这使得代码更具可读性和自解释性。

```
1  from collections import namedtuple
2
3  # 定义一个名为 'Car' 的 namedtuple 类型，它有两个字段：'make' 和 'model'
4  Car = namedtuple('Car', ['make', 'model'])
5
6  # 创建一个 Car 实例
7  my_car = Car(make='Toyota', model='Corolla')
8
9  # 访问元素
10 print(my_car.make)  # 输出: Toyota
11 print(my_car[0])    # 输出: Toyota
12
13 # 尝试修改元素会引发错误，因为 namedtuple 是不可变的
14 # my_car.make = 'Honda'  # ❌ 报错! namedtuple 是不可变的
```

- 优点：**
- 保持元组的轻量与不可变性；
 - 支持字段名访问，提升代码可读性；
 - 可作为简单数据容器替代小型类定义。

应用场景： 配置项、数据库记录、API 返回值等。

实战案例： IrisNet(nn.Module) 中的继承

https://github.com/GMyhf/2025spring-cs201/blob/main/LLM/iris_neural_network.md

来看这个 PyTorch 深度学习模型是如何利用继承的：

```
1 class IrisNet(nn.Module):
2     def __init__(self, input_size=4, hidden_size=10, num_classes=3):
3         super(IrisNet, self).__init__() # ← 关键：调用父类构造函数
4         self.fc1 = nn.Linear(input_size, hidden_size)
5         self.relu = nn.ReLU()
6         self.fc2 = nn.Linear(hidden_size, num_classes)
7
8     def forward(self, x):
9         x = self.fc1(x)
10        x = self.relu(x)
11        x = self.fc2(x)
12        return x
```

继承分析

项目	说明
nn.Module	PyTorch 提供的基类，所有神经网络模块必须继承它
super().__init__()	调用父类初始化逻辑，确保内部机制（如参数注册、设备管理）正常工作
.forward(x)	子类必须实现的方法，定义前向传播流程
self.fc1, self.fc2	使用 nn.Linear 等模块构建网络层，会被自动注册为模型参数

为什么需要继承？

- 1. 统一接口：所有模型都有 .forward() 方法，便于训练循环统一处理；
- 2. 参数管理：nn.Module 自动追踪 self.fc1 这样的子模块，方便优化器更新权重；
- 3. 设备迁移：.to(device) 可一键将整个模型移到 GPU；
- 4. 保存与加载：torch.save(model.state_dict(), ...) 依赖于模块化结构；
- 5. 扩展性强：你可以创建 DeepIrisNet 继承 IrisNet 并添加更多层。

示例进阶：使用继承扩展模型

```
1 class DeepIrisNet(IrisNet): # 继承自 IrisNet
2     def __init__(self, input_size=4, hidden_size=20, num_classes=3,
3 dropout_rate=0.3):
4         super(DeepIrisNet, self).__init__(
5             input_size=input_size,
6             hidden_size=hidden_size,
```

```

6         num_classes=num_classes
7     )
8     # 新增一层和 Dropout
9     self.fc3 = nn.Linear(hidden_size, hidden_size // 2)
10    self.relu2 = nn.ReLU()
11    self.dropout = nn.Dropout(dropout_rate)
12    self.fc4 = nn.Linear(hidden_size // 2, num_classes)
13
14    def forward(self, x):
15        x = super().forward(x) # 复用父类的前两层
16        x = self.fc3(x)
17        x = self.relu2(x)
18        x = self.dropout(x)
19        x = self.fc4(x)
20        return x

```

👉 这体现了继承的强大之处：在不修改原类的前提下进行功能增强。

总结：继承的核心价值

优势	说明
✅ 代码复用	避免重复编写相似代码
✅ 层次清晰	构建类之间的逻辑关系，易于理解和维护
✅ 扩展灵活	子类可在继承基础上自由添加/覆盖行为
✅ 多态支持	同一接口调用不同子类对象，实现动态绑定
✅ 框架兼容	如 PyTorch、Django 等框架大量依赖继承机制

```

1  import torch
2  import torch.nn as nn
3  import torch.optim as optim
4  from torch.utils.data import TensorDataset, DataLoader
5  from sklearn.datasets import load_iris
6  from sklearn.model_selection import train_test_split
7  from sklearn.preprocessing import StandardScaler
8
9  # 1. 加载数据
10 iris = load_iris()
11 X = iris.data
12 y = iris.target
13
14 # 2. 划分训练集和测试集（注意这里先划分再标准化）
15 X_train, X_test, y_train, y_test = train_test_split(
16     X, y, test_size=0.2, random_state=42, stratify=y
17 )
18 """

```

```

19 random_state=42
20 设定随机数种子，从而确保每次运行代码时数据划分的结果都是相同的。这样做可以使实验具有可重复
    性，
21 有利于调试和结果对比。
22
23 stratify=y
24 这个参数表示按照 y 中的标签进行分层抽样，也就是说，训练集和测试集中各类别的
25 比例会与原始数据中的类别比例保持一致。这对于类别不平衡的数据集尤为重要，可以
26 避免某一类别在划分时被严重低估或过采样。
27 """
28
29 # 3. 数据标准化：只在训练集上计算均值和标准差，再将相同的变换应用到测试集上
30 scaler = StandardScaler()
31 X_train = scaler.fit_transform(X_train)
32 X_test = scaler.transform(X_test)
33
34 # 转换为 PyTorch 的 tensor
35 X_train = torch.tensor(X_train, dtype=torch.float32)
36 X_test = torch.tensor(X_test, dtype=torch.float32)
37 y_train = torch.tensor(y_train, dtype=torch.long)
38 y_test = torch.tensor(y_test, dtype=torch.long)
39
40 # 构造数据集和 DataLoader
41 train_dataset = TensorDataset(X_train, y_train)
42 train_loader = DataLoader(train_dataset, batch_size=16, shuffle=True)
43
44
45 # 4. 定义模型
46 class IrisNet(nn.Module):
47     def __init__(self, input_size=4, hidden_size=10, num_classes=3):
48         super(IrisNet, self).__init__()
49         self.fc1 = nn.Linear(input_size, hidden_size)
50         self.relu = nn.ReLU()
51         self.fc2 = nn.Linear(hidden_size, num_classes)
52
53     def forward(self, x):
54         x = self.fc1(x)
55         x = self.relu(x)
56         x = self.fc2(x)
57         return x
58
59
60 model = IrisNet()
61
62 # 5. 定义损失函数和优化器
63 criterion = nn.CrossEntropyLoss()
64 optimizer = optim.Adam(model.parameters(), lr=0.01)
65
66 # 6. 训练模型
67 num_epochs = 100
68 for epoch in range(num_epochs):
69     model.train() # 切换到训练模式
70     running_loss = 0.0
71     for batch_X, batch_y in train_loader:
72         optimizer.zero_grad() # 清空梯度

```

```

73         outputs = model(batch_X)          # 前向传播
74         loss = criterion(outputs, batch_y) # 计算损失
75
76         loss.backward()                    # 反向传播
77         optimizer.step()                   # 更新权重
78
79         running_loss += loss.item() * batch_X.size(0)
80
81         epoch_loss = running_loss / len(train_loader.dataset)
82         if (epoch + 1) % 10 == 0:
83             print(f"Epoch [{epoch + 1}/{num_epochs}], Loss: {epoch_loss:.4f}")
84
85     # 7. 在测试集上评估
86     model.eval() # 切换到评估模式
87     with torch.no_grad(): # 禁用梯度计算，加快测试速度，减少内存消耗
88         outputs = model(X_test)
89         _, predicted = torch.max(outputs, 1)
90         accuracy = (predicted == y_test).sum().item() / len(y_test)
91         print(f"Test Accuracy: {accuracy * 100:.2f}%")
92
93     # 最终预测示例
94     sample = X_test[0].unsqueeze(0) # 取第一个测试样本
95     prediction = torch.argmax(model(sample), dim=1)
96     print(f"\nSample prediction: True class {y_test[0].item()}, "
97           f"Predicted class {prediction.item()}")

```

PyTorch 的 `nn.Module` 基类实现了 `__call__` 方法，当你像函数一样调用模型（`model(input)`）时，`__call__` 会自动处理以下流程：

1. 执行必要的前置操作（如启用训练/评估模式）。
2. 调用你定义的 `forward()` 方法。
3. 返回 `forward` 的输出。

⚠ 你不应该直接调用 `model.forward()`，而应该使用 `model(input)`，因为 `__call__` 中还包含一些重要的钩子（hooks）和状态管理。

结语

继承不仅是语法技巧，更是一种**思维方式**——通过建立类之间的层级关系，能更好地组织复杂系统，提升代码的可维护性和可扩展性。无论是设计数据结构还是搭建深度学习模型，掌握继承都是迈向高级编程的重要一步。

“学会继承，就是学会了站在巨人的肩膀上编程。”

附录

Python数据结构与算法分析（第3版），<https://runestone.academy/ns/books/published/pythonds3/index.html>

算法导论 第三版 (Thmos.H.Cormen ,Charles E. Leiserson etc.)

<https://excalidraw.com>